



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”
Campus de São José do Rio Preto

REDES DE COMPUTADORES (I)

Redes de Computadores I (1960SCC) e Redes de Computadores (2375SCC) – 2024

Prof. Dr. Adriano Mauro Cansian

adriano.cansian@unesp.br

[linkedin.com/in/adrianocansian/](https://www.linkedin.com/in/adrianocansian/)



Curso de Redes de Computadores

Prof. Dr. Adriano Mauro Cansian

Capítulo 3 Camada de Transporte (Parte 1)

Licença de uso deste material (1)



Estes slides foram produzidos e são disponibilizados sob a licença ["Creative Commons Atribuição Não Comercial SemDerivações 4.0 Internacional \(CC BY-NC-ND 4.0\)"](https://creativecommons.org/licenses/by-nc-nd/4.0/).

Como dar o crédito apropriado:

Para qualquer reutilização ou distribuição, você deve deixar claro a terceiros os termos da licença a que se encontra submetida esta obra, e seu autor. A melhor maneira de fazer isso é citando a fonte e contato. Por exemplo:

Fonte: Adriano Mauro Cansian. "Redes de Computadores I – Camada de Transporte – Parte 1", São Paulo, SP. (2025).

Contato: adriano.cansian@unesp.br. Licenciamento Creative Commons [\(CC BY-NC-ND 4.0\)](https://creativecommons.org/licenses/by-nc-nd/4.0/)

Fair Use: Este material pode usar citações e figuras de outros autores. Direitos reservados aos devidos autores. Todas as citações feitas em caráter didático.

Avisos: Você não tem de cumprir com os termos da licença relativamente a elementos do material que estejam no domínio público ou cuja utilização seja permitida por uma exceção ou limitação que seja aplicável.

Não são dadas quaisquer garantias. A licença pode não lhe dar todas as autorizações necessárias para o uso pretendido. Por exemplo, outros direitos, tais como direitos de imagem, de privacidade ou direitos morais, podem limitar o uso do material.

Licença de uso deste material (2)



- **Você tem o direito de:**

Compartilhar - copiar e redistribuir o material em qualquer suporte ou formato.

- **De acordo com os termos seguintes:**

- **(BY) Atribuição** - Você deve dar o crédito apropriado, prover um *link* para esta licença e indicar se mudanças foram feitas. Você deve fazê-lo em qualquer circunstância razoável, mas de nenhuma maneira que sugira que o licenciante apoia você ou o seu uso.
- **(NC) Não Comercial** - Você não pode usar o material para fins comerciais.
- **(ND) Sem Derivações** - Se você remixar, transformar ou criar a partir do material, você não pode distribuir o material modificado.
- **Sem restrições adicionais** - Você não pode aplicar termos jurídicos ou medidas de caráter tecnológico que restrinjam legalmente outros de fazerem algo que a licença permita.

Referência para estudo:

REDES DE COMPUTADORES E A INTERNET - 6ª Edição

James F. Kurose e Keith W. Ross. Edit. Pearson 2014.

ISBN: 9788581436777.

Capítulo 3 – **Camada de Transporte** (6a. Ed.)



Capítulo 3: Camada de Transporte

Metas do capítulo:

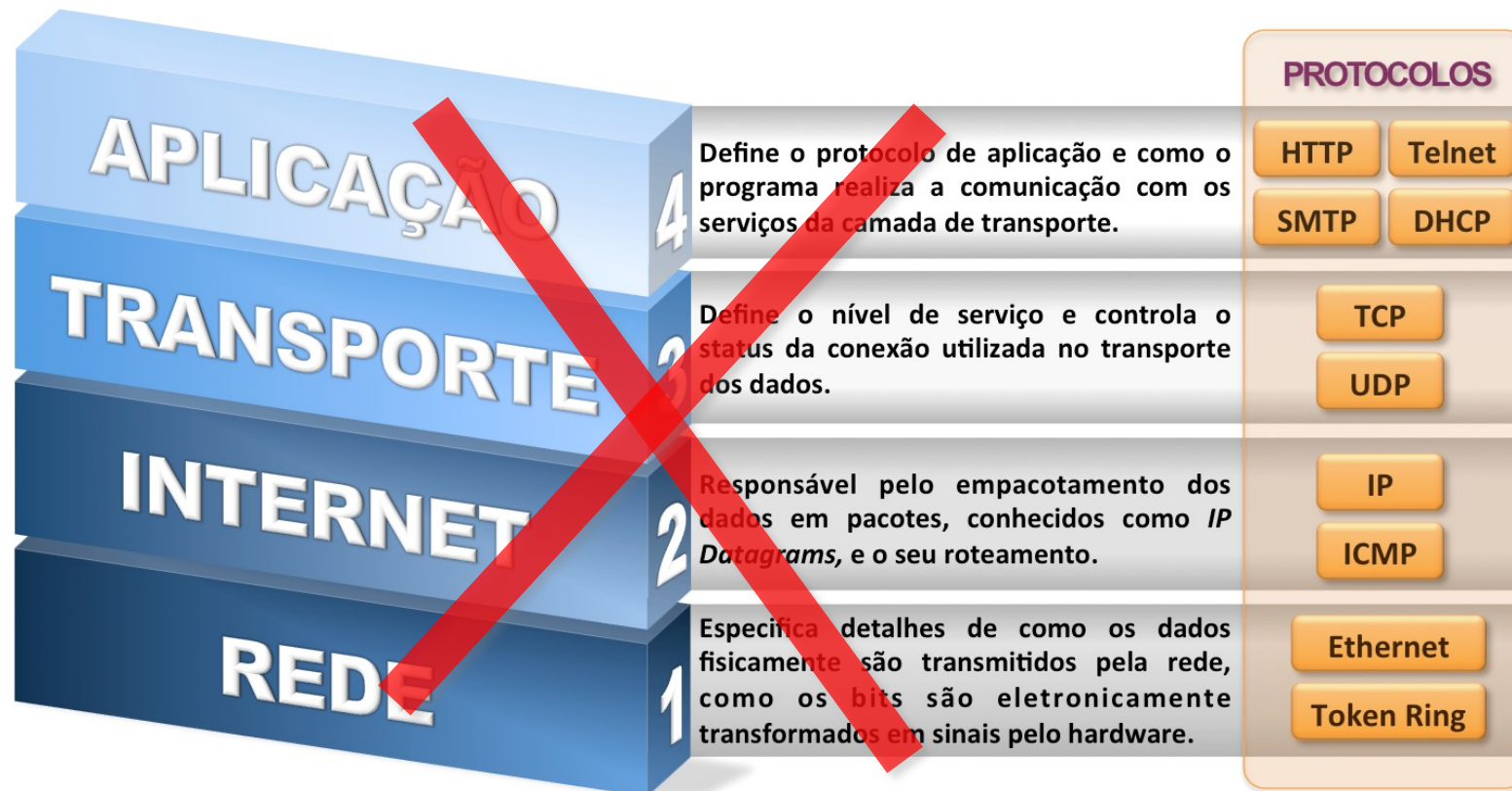
- ❑ Compreender os princípios dos serviços da camada de transporte.
- ❑ Entender o **UDP**.
- ❑ Entender o **TCP**.

O que veremos:

- ❑ Serviços da camada de transporte.
- ❑ Multiplexação / desmultiplexação.
- ❑ Transporte sem conexão: UDP.
- ❑ Princípios de transferência confiável.
 - Qual a ideia e como surgiu.
- ❑ Transporte orientado a conexão: TCP
 - Transferência confiável.
 - Controle de fluxo.
 - Gerenciamento de conexões.
 - Princípios de controle de congestionamento.
 - Controle de congestionamento em TCP.

Cuidado com conceitos !

Muito material errado ou ruim na Internet!
(apesar de outros muito bons)



Comparação entre as camadas

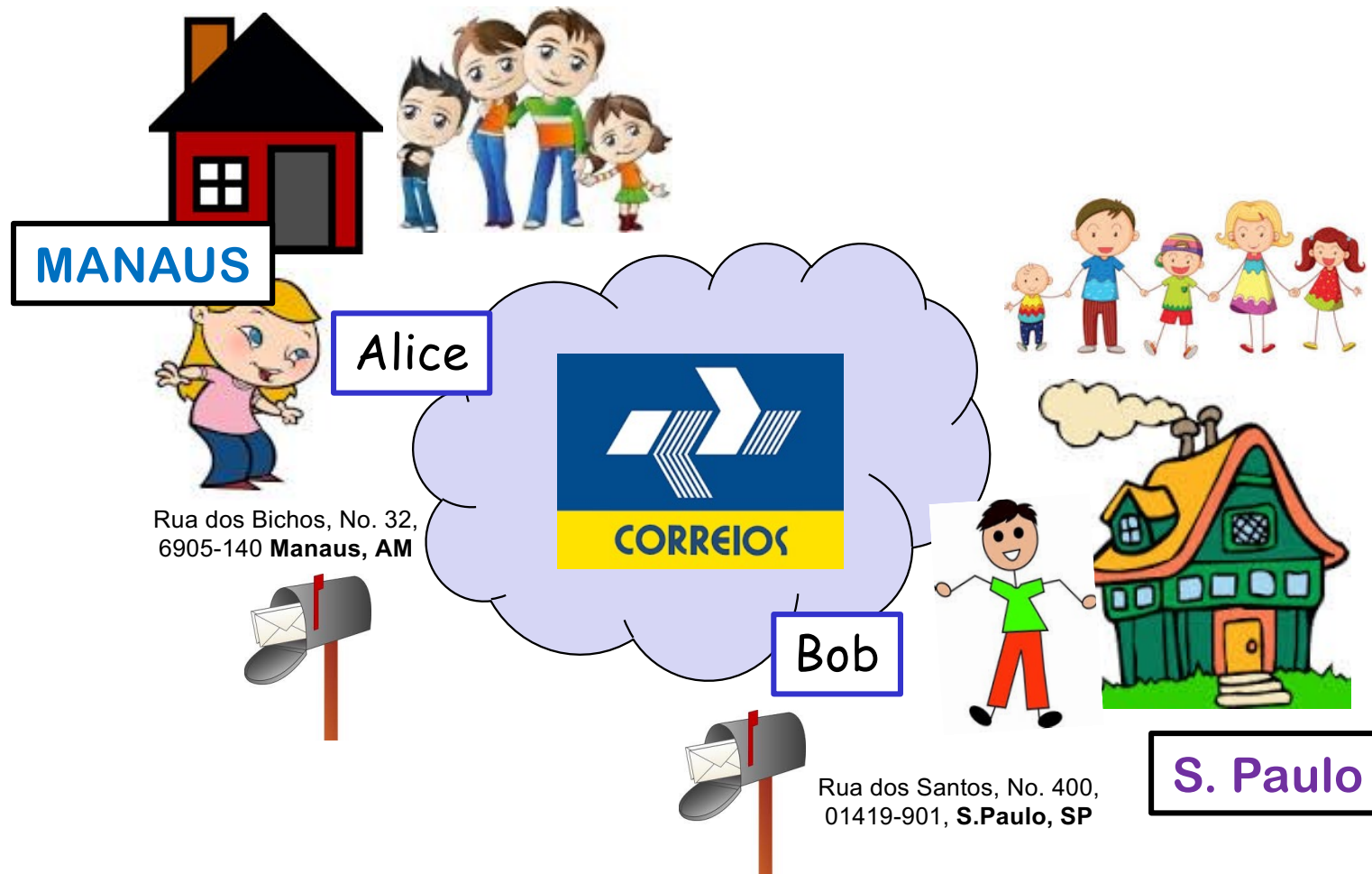
Camada **Transporte** → interliga **processos**

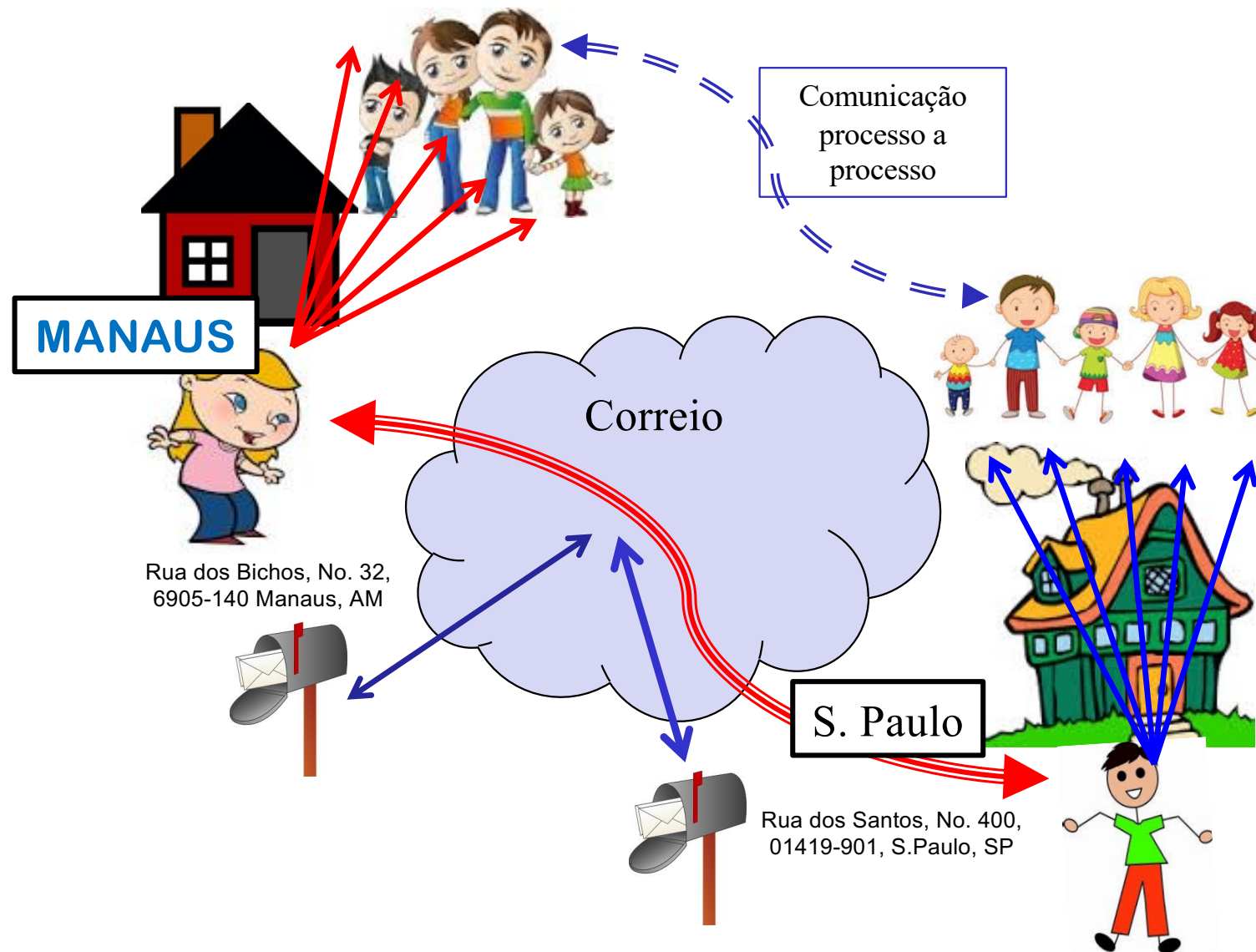
Camada de **rede** → interliga **hosts**

(será vista no próximo capítulo)

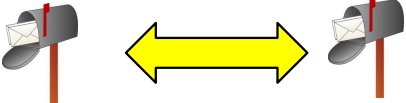
Esta distinção é sutil, mas **MUITO** importante.

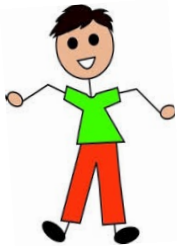
Analogia entre camadas





Camada de Transporte X Camada de Rede

- ❑ **Hosts** = Casas. 
- ❑ **Processos** = pessoas que trocam mensagens. 
- ❑ Mensagens da aplicação = cartas em envelopes. 
- ❑ **Protocolo da camada de REDE:**
 - **Serviço postal** (correio). 
- ❑ **Protocolo da Camada de TRANSPORTE:**
 - **João** (de um lado) e **Maria** (do outro).

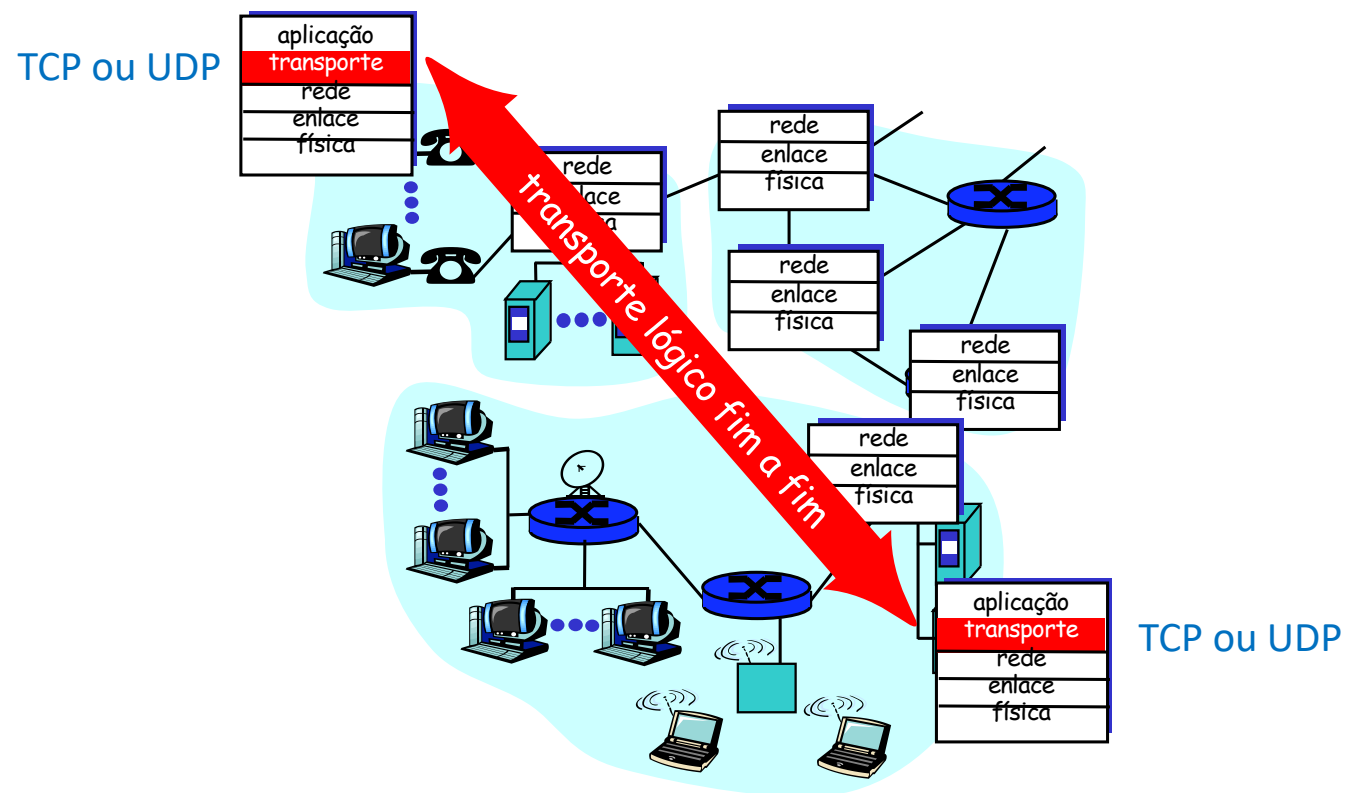


Serviços e protocolos de TRANSPORTE (1)

Do ponto de vista da **APLICAÇÃO**:

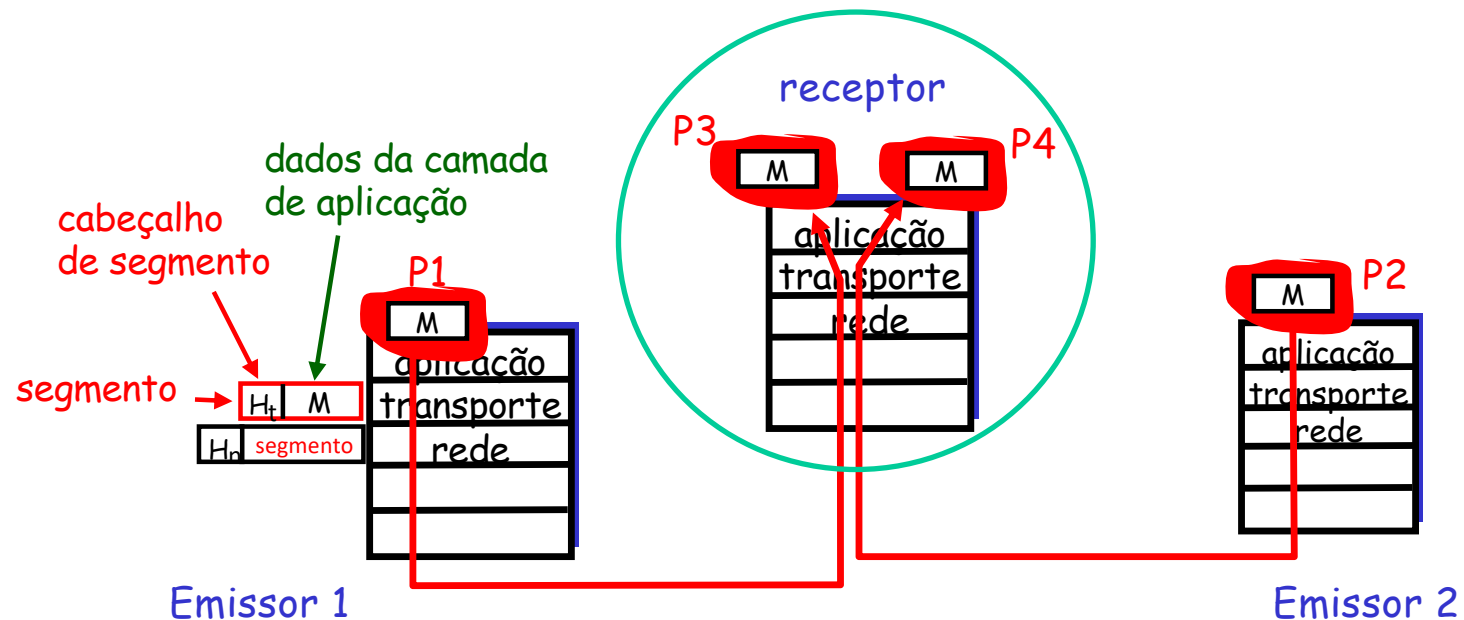
A **camada de transporte** é quem permite a camada de **aplicação** enxergar os sistemas **como se eles estivessem fisicamente conectados.**

Serviços e protocolos de TRANSPORTE (2)

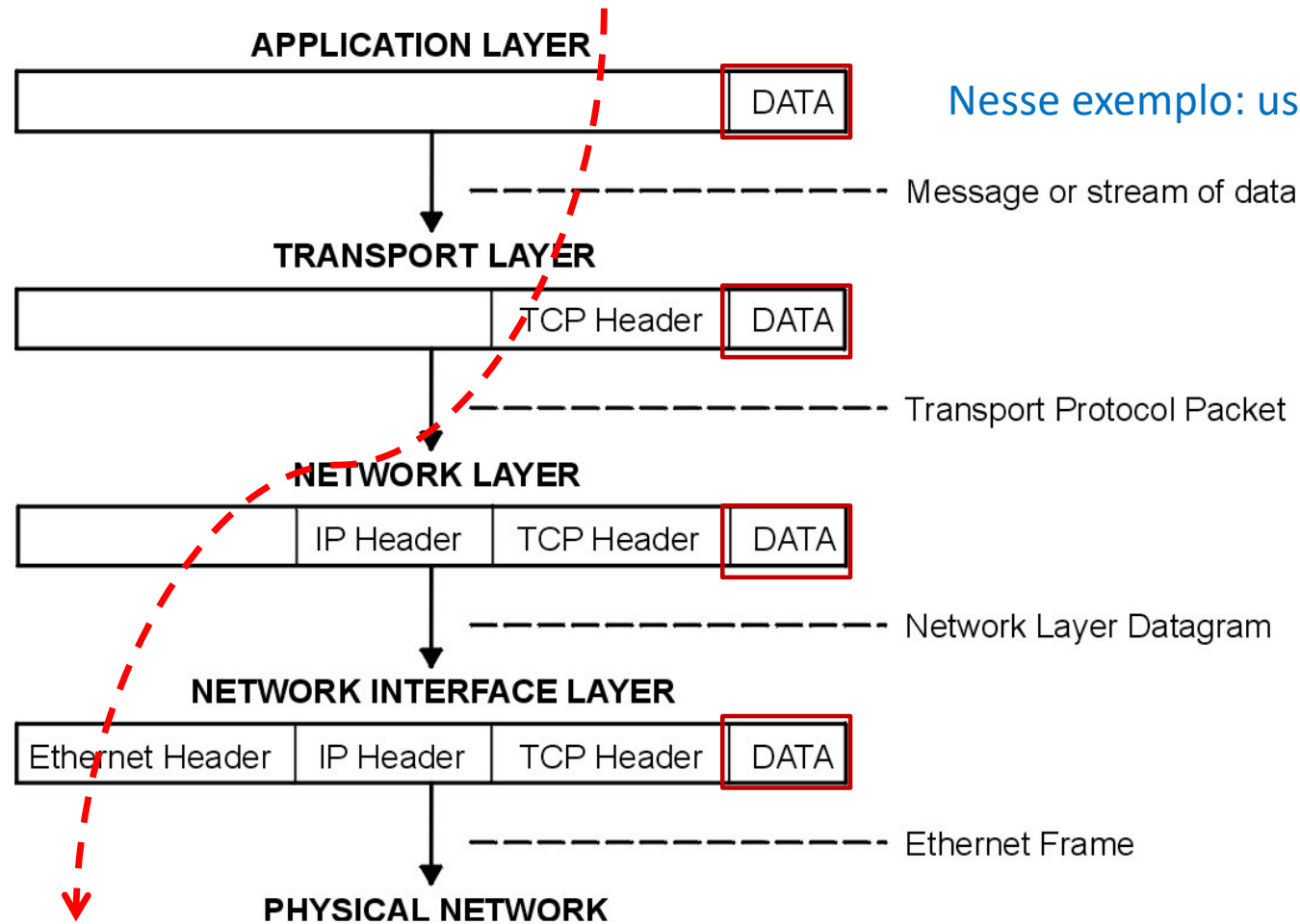


Multiplexação/desmultiplexação

- ❑ **MULTIPLEXAÇÃO**: Juntar dados de múltiplos processos de aplicações, **envelopando** dados com **cabeçalho**.
- ❑ **DEMULTIPLEXAÇÃO**: entrega de segmentos recebidos para os processos da camada de aplicação corretos.

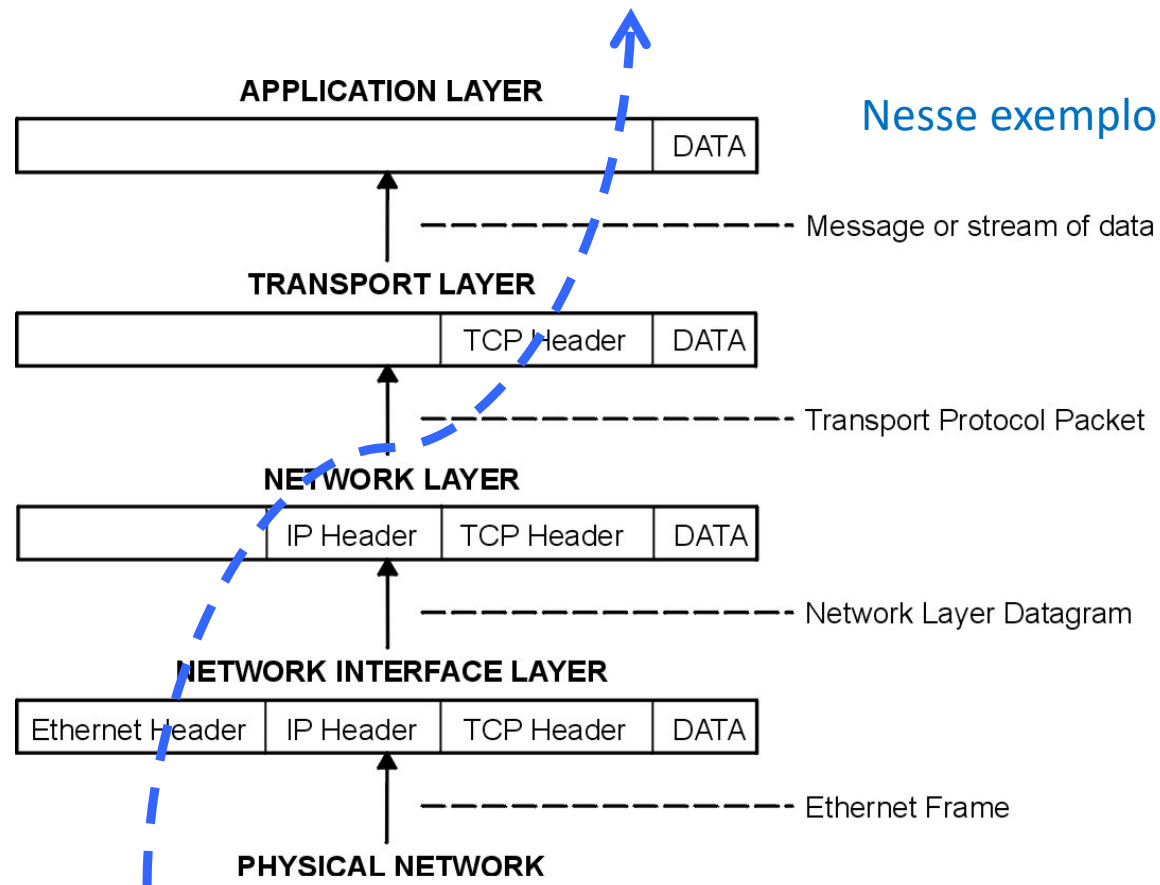


Fluxo de encapsulamento



Nesse exemplo: usando TCP

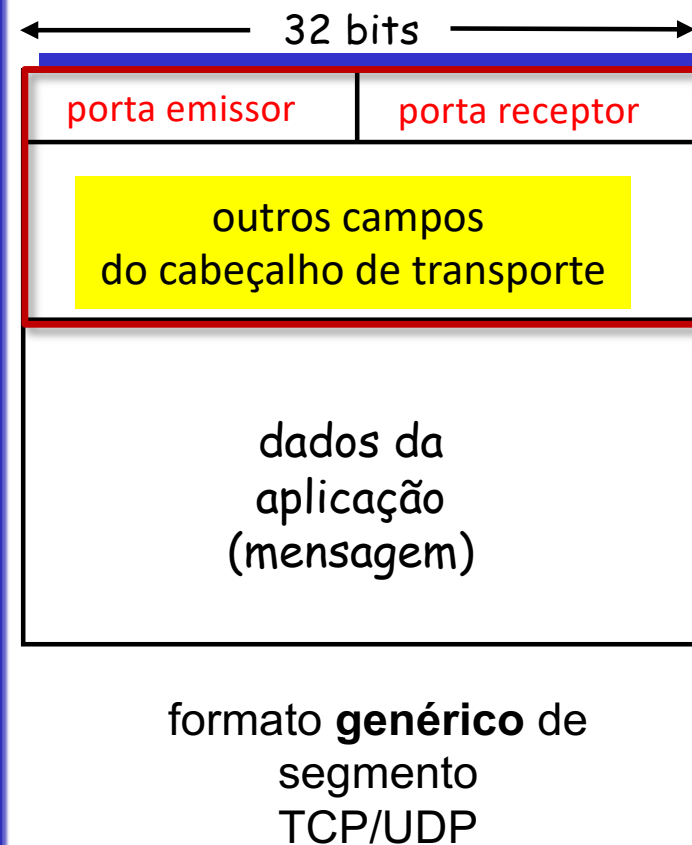
Fluxo de desencapsulamento



Portas são fundamentais

Multiplex / demultiplex:

- ❑ Baseadas em **números de porta** e **endereços IP** de remetente e receptor:
 - Rever: portas WKS e portas efêmeras do capítulo anterior.



Os protocolos de Transporte

UDP

User Datagram Protocol

UDP: User Datagram Protocol [RFC 768]

- ❑ Protocolo de transporte simples.
- ❑ Serviço “**melhor esforço**”.
 - Resulta que segmentos UDP possam ser:
 - **Perdidos**.
 - Entregues **fora de ordem** à aplicação.
- ❑ Sem conexão:
 - **Não há “setup”** entre remetente e receptor.
 - **Não há “status”** (é *stateless*).
 - **Sem confirmação** de entrega.
- ❑ Sem controle de congestionamento.
- ❑ Sem controle de fluxo.

UDP: *User Datagram Protocol* [RFC 768]

Por quê existe um UDP?

❑ **Elimina** estabelecimento de **conexão**:

- Reduz *delay* de “partida lenta” → visto mais adiante.

❑ **Simple**s: não mantém “estado”:

- *Stateless*.

❑ **Pequeno cabeçalho** de segmento:

- Mais simples = menos desperdício de dados (*overhead de 8 bytes x 20 bytes*)

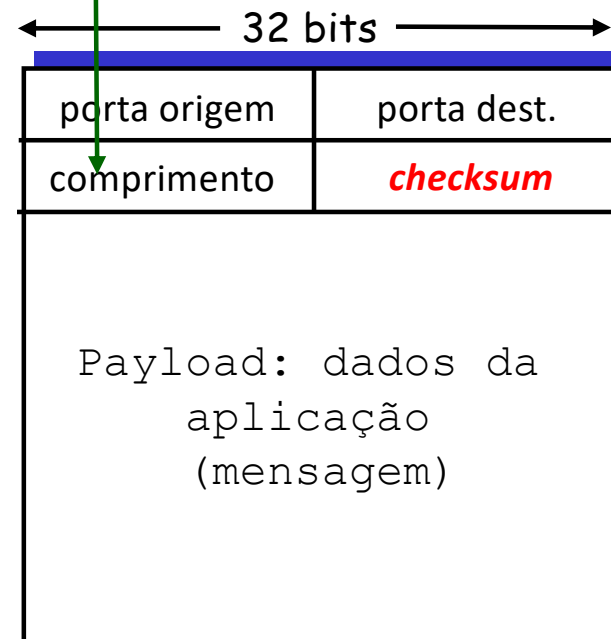
❑ **Sem controle de congestionamento nem de fluxo**:

- UDP pode *transmitir o mais rápido possível que puder*.

Mais sobre UDP

- ❑ Muito utilizado para aplicações de mídia.
 - São tolerantes a perdas.
 - São sensíveis à taxa de transmissão.
- ❑ Outros usos de UDP :
 - DNS (servidor de nomes).
- ❑ Transferência confiável com UDP só é possível se **incluir confiabilidade na camada de aplicação**.
 - Recuperação de erro tem que ficar por conta da aplicação.

Comprimento em bytes do segmento UDP, incluindo cabeçalho



Formato de um segmento UDP

Cálculo do Checksum

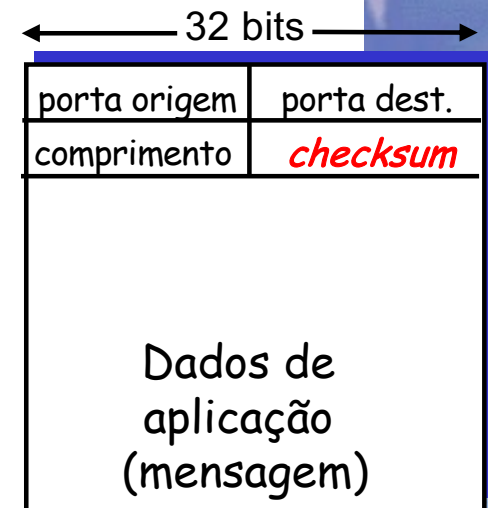
(válido para o UDP e o TCP)

Checksum UDP - EMISSOR

Finalidade: detectar falha no segmento transmitido.

Emissor:

1. Trata o segmento como sequência de inteiros de **16-bits**.
2. *Checksum*: soma (usando complemento de 1) **do segmento**.
3. Para fazer o cálculo no EMISSOR:
 1. Considera o campo *checksum* como zeros.
 2. **Inclui os dados (payload)**.
 3. Inclui um *pseudoheader* no cálculo (**IMPORTANTE! visto adiante**).
4. **EMISSOR coloca complemento de um** do valor da soma no campo *checksum* de UDP .
5. E **envia** o segmento.



Checksum UDP - RECEPTOR

Receptor (na chegada):

1. Soma os campos de **header**, e **dados** do segmento recebido.
 - Incluindo o valor que está no campo *checksum*.
 - Como colocou o **complemento de 1** no campo *checksum*, quando somar tudo tem que **resultar num valor com todos os bits iguais a 1**.
2. Verifica se o **valor RESULTANTE é todo igual a 1**:
 - SIM = nenhum erro detectado (=1).
 - NÃO = erro detectado. ($\neq 1$).

❑ Mas ainda pode haver erros?

(Veremos mais adiante)

Exemplo de cálculo do *checksum* (3)

Pseudo header proveniente do IP
(vide adiante)

153.18.8.105			
171.2.14.10			
All 0s	17	15	
1087		13	
15		All 0s	
T	E	S	T
I	N	G	Pad

Campo "Protocol" da camada de rede:
Decimal 17 indica que carrega um protocolo UDP

"padding" será explicado mais adiante

10011001	00010010	→	153.18
00001000	01101001	→	8.105
10101011	00000010	→	171.2
00001110	00001010	→	14.10
00000000	00010001	→	0 and 17
00000000	00001111	→	15
00000100	00111111	→	1087
00000000	00001101	→	13
00000000	00001111	→	15
00000000	00000000	→	0 (checksum)
01010100	01000101	→	T and E
01010011	01010100	→	S and T
01001001	01001110	→	I and N
01000111	00000000	→	G and 0 (padding)
10010110	11101011	→	Sum
01101001	00010100	→	Checksum
			a ser enviado

Checksum – esquema simplificado

```
0110011001100110
0101010101010101
...
0000111100001111
0000000000000000
```

Emissor

Segmento inteiro
(incluindo dados)

Campo *Checksum* zerado para fins do cálculo.

```
1100101011001010
```

Soma

```
0011010100110101
```

Checksum (transmitido)

```
0110011001100110
0101010101010101
...
0000111100001111
0011010100110101
```

Receptor

Segmento inteiro

Checksum (recebido)

```
1111111111111111
```

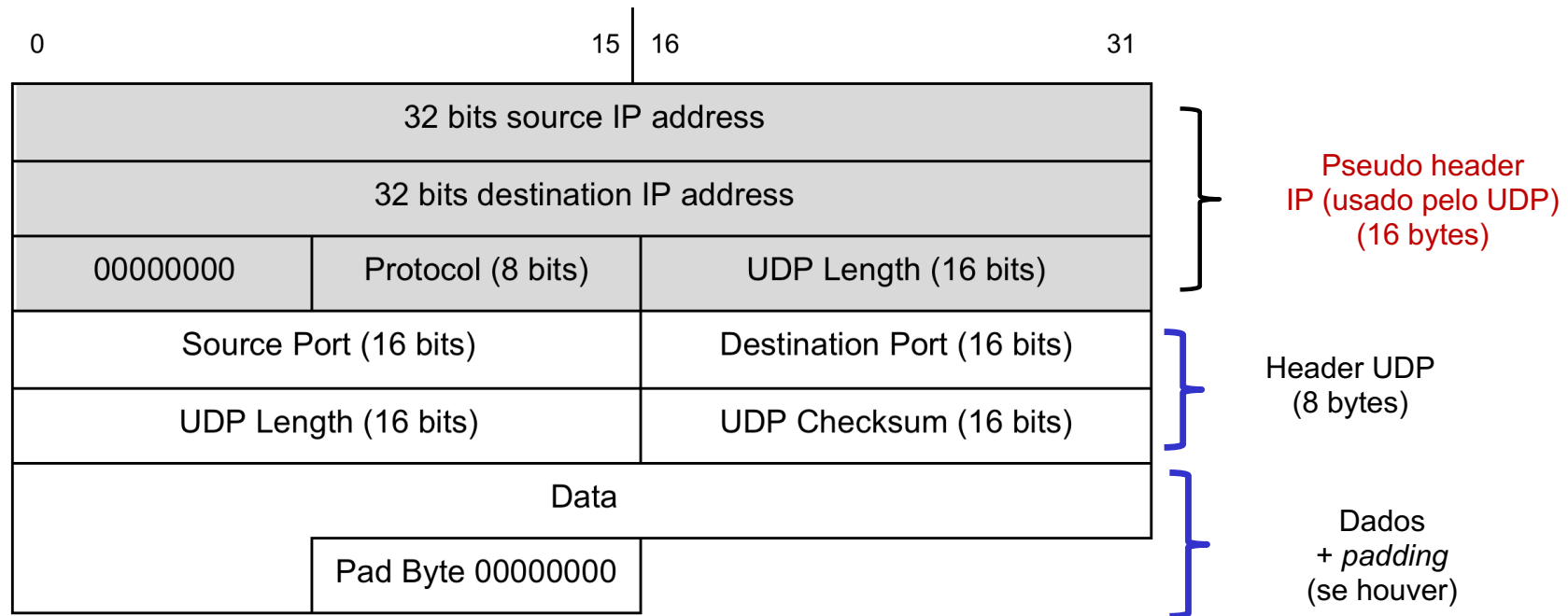
Soma + *checksum*

Se não resultar em all 1's = erro

Pseudo Header (1)

- ❑ TCP e UDP usam um **pseudo header da camada IP** no cálculo do *checksum*.
 - Usa informações da camada IP (16 bytes).
 - *Checksum* usa os *headers* (**pseudo header** e *header* UDP/TCP), e os **dados** do UDP/TCP.
- ❑ Comprimento do UDP pode ser um número ímpar de bytes.
 - Porque contém os **dados** (**payload**).
 - Algoritmo do *checksum* soma blocos de palavras de 16 bits.
 - Solução: adicionar preenchimento (“*padding*”) de zeros, no final, se necessário, **caso o número de blocos de 16 bits seja ímpar**.
 - *Padding* não é transmitido.

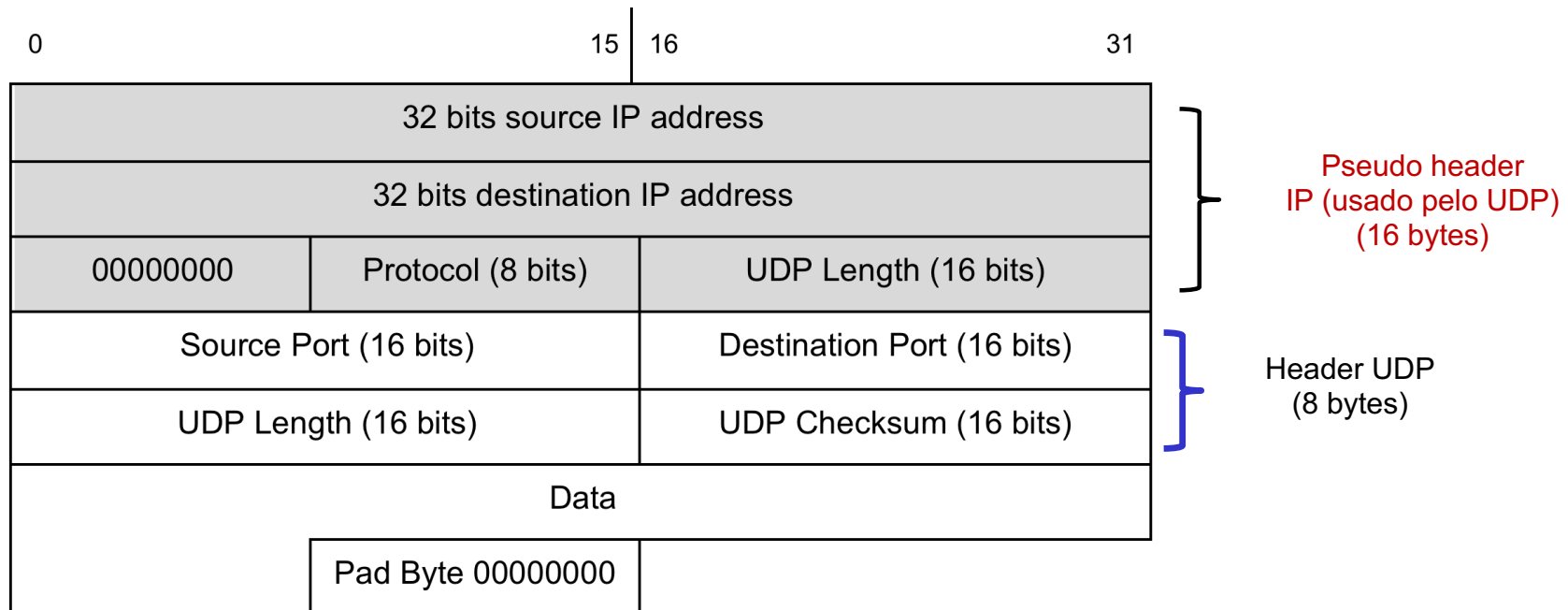
Pseudo Header (2)



Padding usado apenas para garantir que o cálculo tenha um tamanho múltiplo de 16 bits.

- O comprimento dos dados deve ser um múltiplo de 16 bits para garantir a validade do algoritmo de *checksum*.
- Se não for múltiplo de 16 bits, o *padding* é adicionado.
 - Garantir que o comprimento total seja adequado para o cálculo do *checksum*.
- Mas o *padding* em si não é especificamente considerado no cálculo do *checksum* TCP.
- O *padding* não é transmitido. O algoritmo do receptor apenas sabe que, se não for múltiplo de 16 bits, deve somar 16 bits 0s. ²⁸

Violação do princípio de camadas



Explique por que ocorre uma violação da independência das camadas.

Exemplo de cálculo do *checksum* (1)

Ver RFC-1071 !

Considere 3 palavras de 16 bits sendo transmitidas:

```
0110011001100110
0101010101010101
0000111100001111
```

A soma das duas primeiras palavras é:

```
0110011001100110
0101010101010101
1011101110111011
```

Adicionando a terceira palavra, a soma acima fica:

```
1011101110111011
0000111100001111
1100101011001010
```

← soma

- Os complementos de 1 são obtidos convertendo todos os 0's para 1's , e todos os 1's para 0's.

0011010100110101

Exemplo de cálculo do *checksum* (2)

Assim o complemento de 1's da soma 1100101011001010

é 0011010100110101

- Este valor se torna o *checksum*.

0	15	16	31
Source Port Number(16 bits)		Destination Port Number(16 bits)	
Length(UDP Header + Data)16 bits		UDP Checksum(16 bits) 0011010100110101	
Application Data (Message)			

Exemplo de cálculo do *checksum* (2)

Assim o complemento de 1's da soma 1100101011001010

é 0011010100110101



- ❑ Este valor se torna o *checksum*.
- ❑ No receptor, **todas as palavras de 16 bits são somadas**, incluindo o campo *checksum*.
- ❑ Se não foram introduzidos erros no pacote, a soma no receptor tem que resultar em **1111111111111111** uma vez que somou-se ao header o complemento de 1 da soma dele.
- ❑ Se pelo menos um dos bits for zero, então algum erro foi introduzido no pacote.

Pergunta exercício: Por que o UDP usa *checksum*, se a maioria dos protocolos *data-link* (inferiores), incluindo o popular Ethernet, também fornece verificação de erro ??

Lembrete 1: método Polinomial

Modo Polinomial (“vai um”)

(Este é o método realmente usado pelo UDP. Em caso de dúvida, consulte a seção 3.3.2 do livro texto do curso – “*Computer Networking: A Top-Down Approach Featuring the Internet*” - James F. Kurose & Keith W. Ross).

Lembrar que:

$$0 + 0 = 0 = 00 \text{ (“vai zero”)}$$

$$1 + 0 = 1 = 01 \text{ (“vai zero”)}$$

$$0 + 1 = 1 = 01 \text{ (“vai zero”)}$$

$$0 + 1 = 1 = 01 \text{ (“vai zero”)}$$

$$1 + 1 = 2 = 10 \text{ (“vai 1”)}$$

$$1 + 1 + 1 = 3 = 11 \text{ (“vai 1”)}$$

$$\begin{array}{r}
 0^1 \ 1^1 \ 0^1 \ 1 \ 0 \ 1 \ 0 \ 1 \\
 + \ 0 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \\
 \hline
 1 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1
 \end{array}$$

$$\begin{array}{r}
 1^1 \ 1 \ 0 \ 0^1 \ 0^1 \ 1 \ 0 \ 1 \\
 + \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 0 \\
 \hline
 1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1
 \end{array}$$

Complemento de 1 = 0 1 1 0 1 1 1 0

Lembrete 2: “carry”

Na aritmética de complemento de 1, dois números são somados da mesma forma que na representação binária. Com a diferença que, **na ocorrência de transporte (carry – vai um) na soma parcial dos bits mais à esquerda, este transporte será somado ao resultado.**

Exemplo: somar os valores 10 e – 3 em complemento de 1, para 8 bits

10 em complemento de 1 é 00001010 (10)

- 3 em complemento de 1 é 11111100 (-3)

somando

$$\begin{array}{r} 10000110 \\ + 11111100 \\ \hline 10000110 \end{array}$$
 1
 carry

Observe que houve carry. Este carry deve ser somado ao resultado obtido. Vejamos:

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

00000110

+ 1

00000111 (7) que é o resultado da operação.

Exemplo de soma de verificação da Internet

□ nota

- Ao somar números, um carryout do bit mais significativo precisa ser somado ao resultado

□ exemplo: somar dois inteiros de 16 bits

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

contorna	1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

soma	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
soma de	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1
verificação	

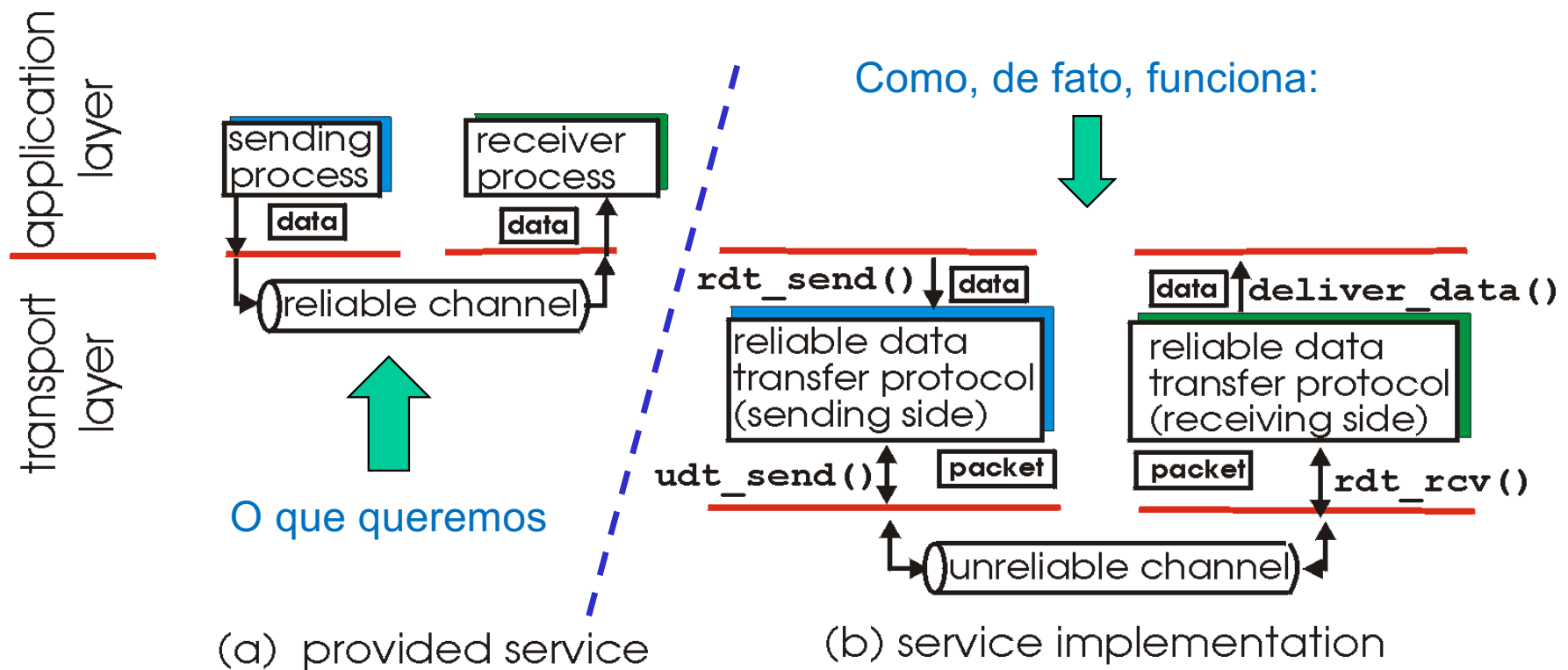
Note: A red circle highlights the first '1' in the 'contorna' row, and a red arrow points from it to the right, indicating the carryout.

Princípios de Transferência Confiável de Dados

Reliable Data Transfer (RDT)

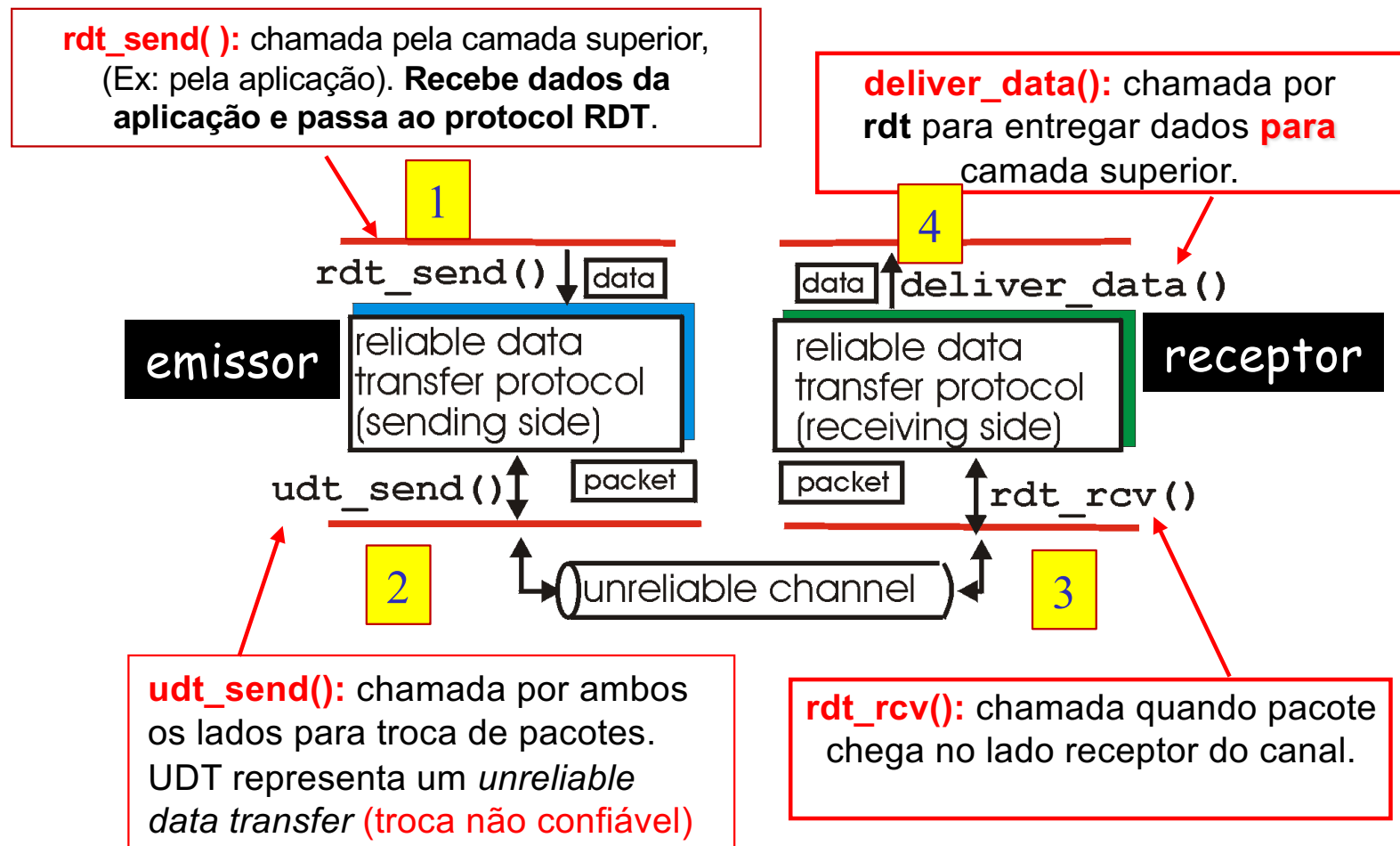
Cenário para estudar *Reliable Data Transfer* (RDT)

Queremos tornar um canal não confiável num canal confiável.



Características do canal não confiável determinam a complexidade de um protocolo de transferência confiável de dados (RDT).

Transferência confiável de dados (RDT) Como começar?



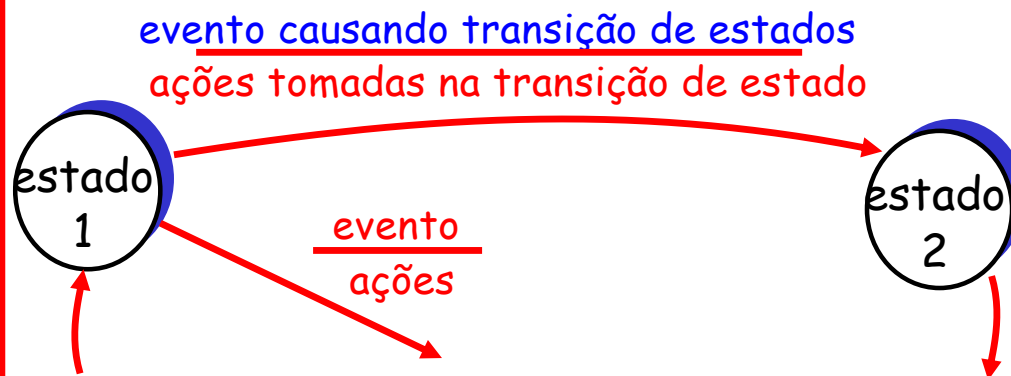
Observação: **Consulte o apêndice para entender melhor as funções RDT.**

Transferência confiável de dados (rdt): como começar

Iremos:

- ❑ Desenvolver passo-a-passo os lados **remetente** e **receptor** de um protocolo confiável RDT.
- ❑ Vamos Considerar apenas fluxo unidirecional de dados.
 - Mas a informação de controle flui em ambos sentidos!
- ❑ Usar máquinas de estados finitos (FSM - *Finite State Machine*) para especificar **remetente** e **receptor**.

ESTADO: quando o sistema está num “estado”, o **próximo estado** é determinado unicamente pelo **próximo evento**.



Como iremos analisar:

□ Para entender a evolução das ideias:

- Vamos fazer considerações sobre **características que podem existir no canal.**
 - Exemplos: canal pode entregar dados fora de ordem, inverter bits, perder bits, dentre outros.

□ Importante: o **RDT ainda não é o TCP.**

- Mas é de onde ele veio.
- Trata-se de como as ideias foram surgindo, e como o TCP evoluiu.

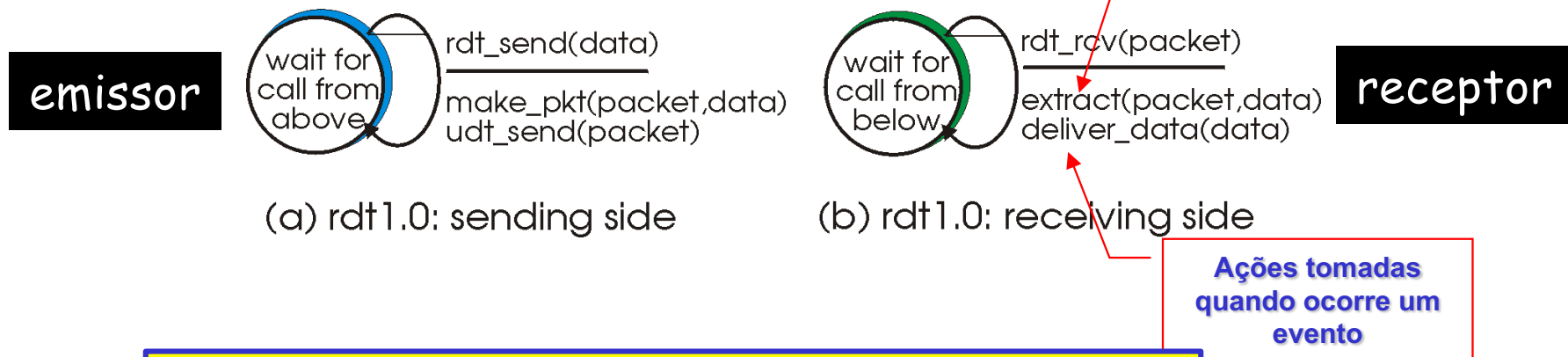
RDT 1.0: transferência confiável usando um canal confiável

❑ Suposição 1.0: o canal é perfeitamente confiável.

- Não apresenta erros de bits.
- Não apresenta perda de pacotes.

❑ FSMs separadas, para remetente e receptor:

- Remetente envia dados pelo canal.
- Receptor recebe dados do canal.



Observação: este tipo de canal não existe!!

RDT 2.0 - Modelo um pouco mais realista:

- ❑ Bits num pacote **podem ser corrompidos**.
 - Devido a falhas nos dispositivos físicos da rede.
 - Por exemplo, quando um pacote é transmitido, **ocorre inversão de 1 bit**.
- ❑ Entretanto, continuamos supondo que **todos** os pacotes transmitidos são **recebidos na ordem** em que são enviados.
 - **Não há PERDA** de pacotes.
 - Ainda que bits possam estar **corrompidos**.
 - Significa o zero pode se tornar 1 e/ou o 1 pode se tornar zero.

Rdt2.0: canal com erros de bits

- ❑ Uma vez que o canal pode inverter bits no pacote:
 - O *checksum* pode detectar erros de bits.
- ❑ A questão é: como recuperar dos erros?



Rdt2.0: canal com erros de bits

□ Como recuperar dos erros?

- Uma boa ideia: usar **mensagens de controle**.
- *Confirmação (ACK)*: receptor avisa ao remetente que pacote chegou bem.
- *Negativa de Confirmação (NAK)*: receptor avisa explicitamente ao remetente que pacote tinha erros.
 - Emissor retransmite pacote ao receber um NAK

(Lembrar de cenários humanos usando ACKs, NAKs)

Mensagens de controle

- ❑ Receptor deve informar ao emissor o que foi recebido certo.
- ❑ E aquilo que foi recebido com erro.
 - Exigindo retransmissão.
- ❑ Protocolos baseados em retransmissão são chamados de Protocolos **ARQ** - *Automatic Repeat reQuest*.
 - Requisição Automática de Repetição.

Novas capacidades em RDT 2.0

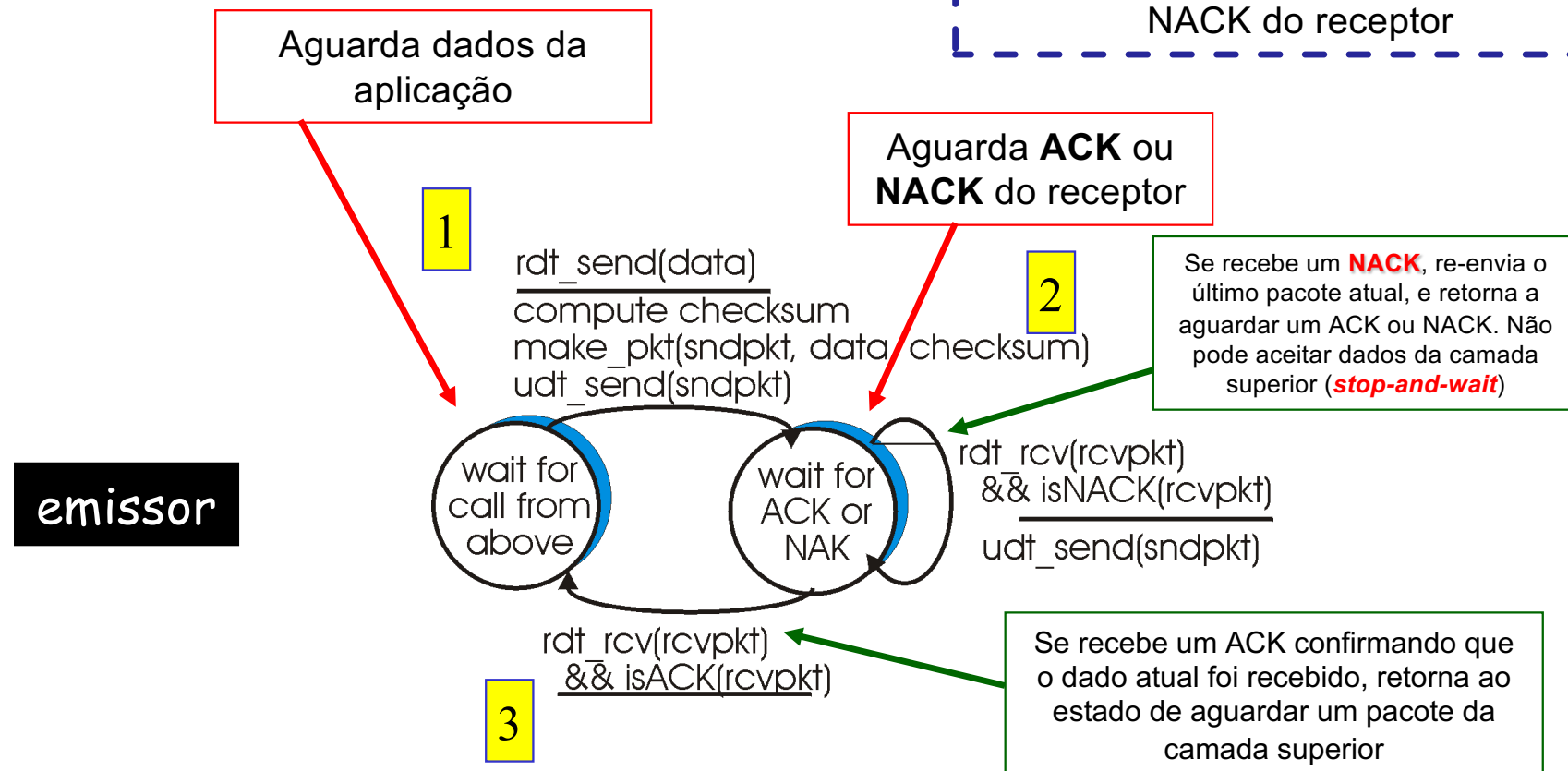
Três **capacidades adicionais** são necessárias:

1. **Detecção de erros:** para permitir que o receptor **identifique** erros de bit.
2. **Realimentação (*feedback*) pelo receptor:** envio de **mensagens de controle** (ACK ou NAK) entre **receptor** e o **emissor**.
3. **Retransmissão:** para **corrigir** erros detectados.

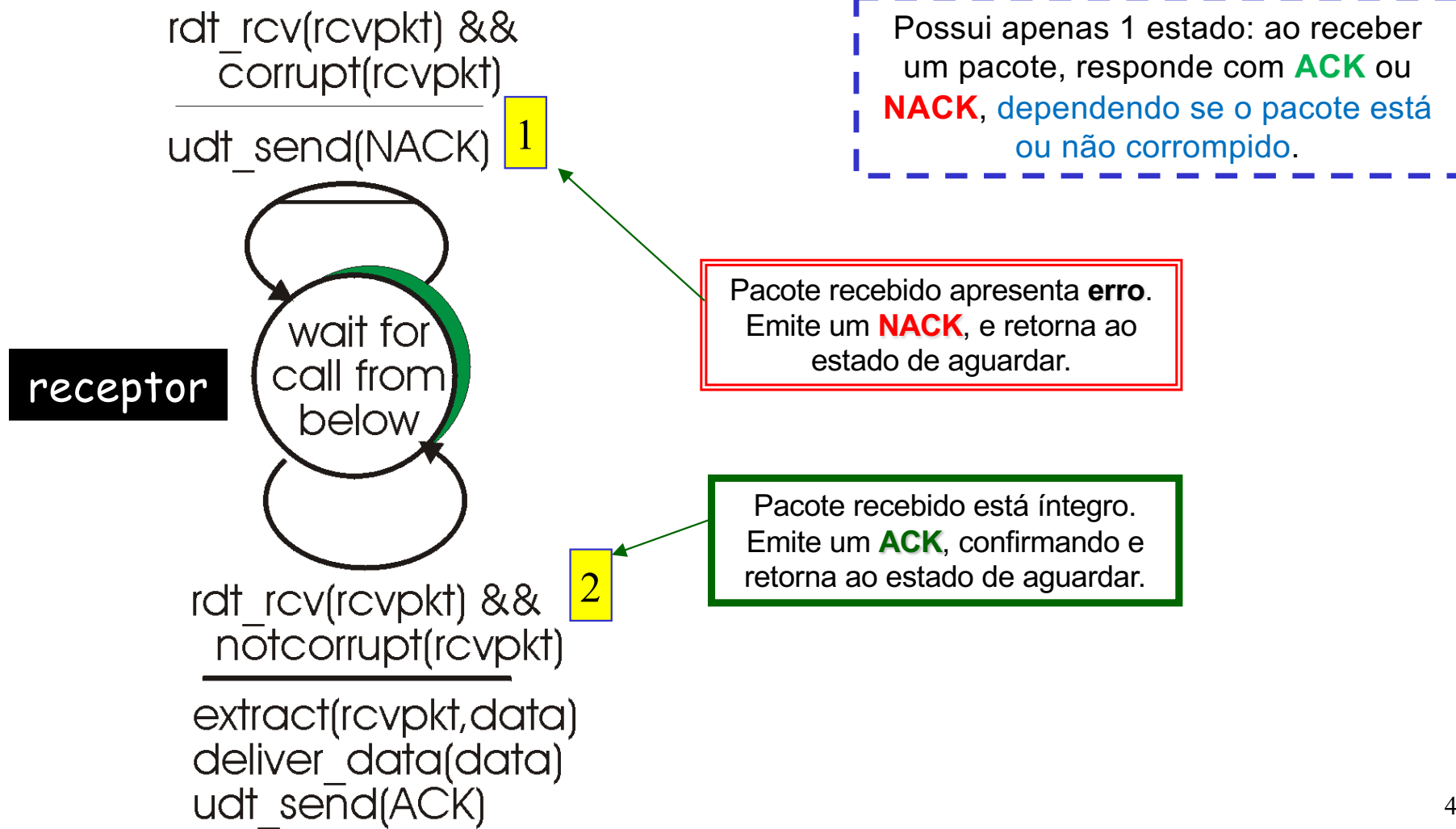
Esses são os novos mecanismos que precisam estar presentes na proposta de protocolo **rdt 2.0**

rdt2.0: especificação da FSM - Emissor

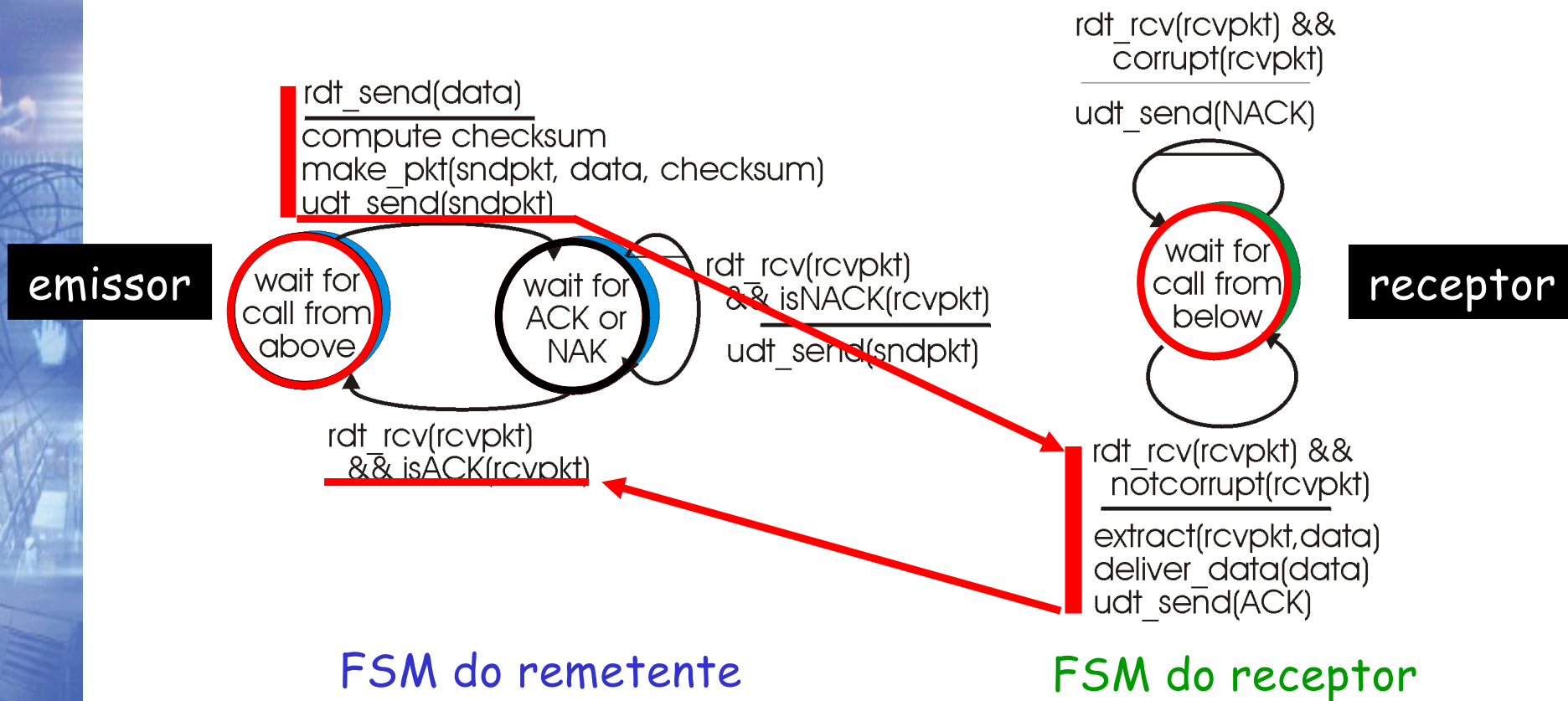
Possui 2 estados: Em um estado aguarda dados da camada superior. Em outro estado, aguarda ACK ou NACK do receptor



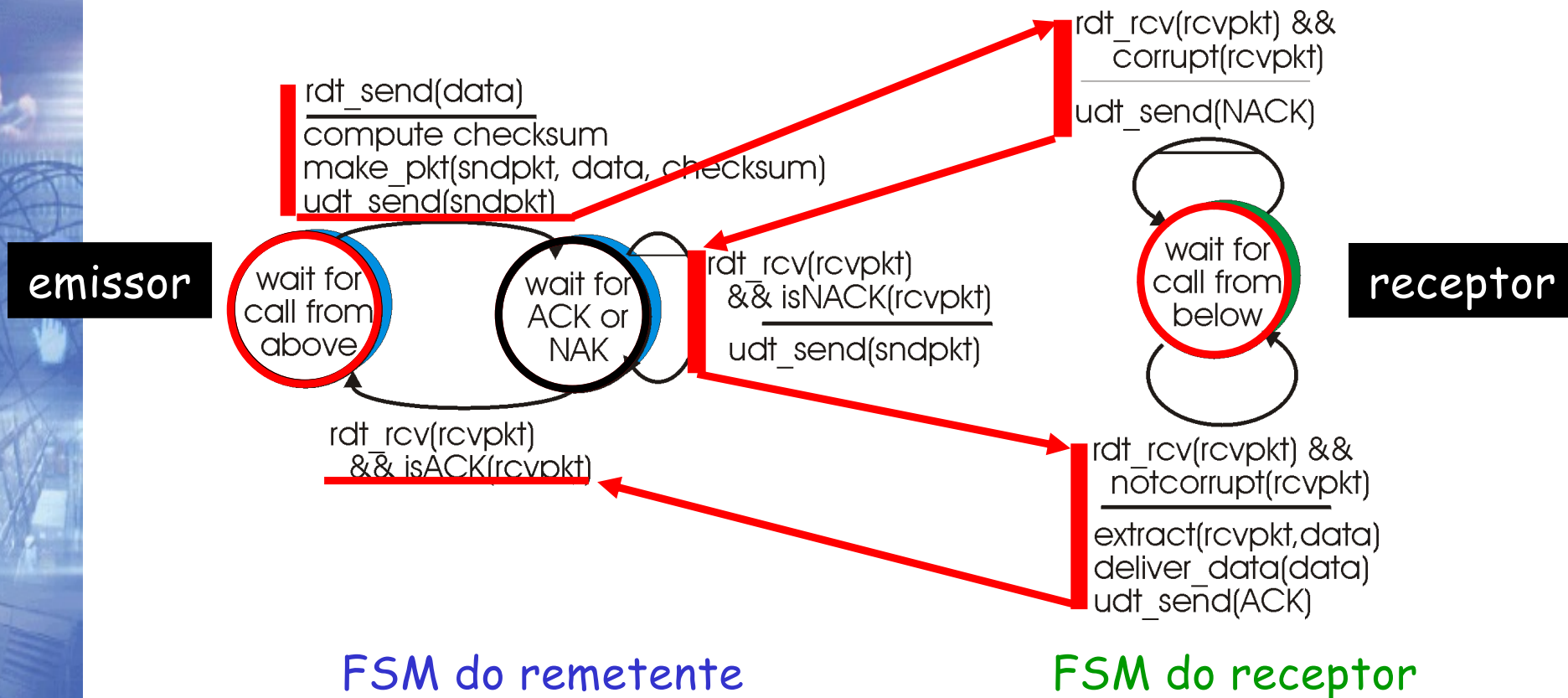
rdt2.0: especificação da FSM - Receptor



rdt2.0: em ação (sem erros)



rdt2.0: em ação (cenário de erro)



Mas, o RDT 2.0 tem algum problema?

RDT 2.0 tem uma falha fatal:

- ❑ O que acontece se ACK ou NACK **com erro**?
 - **Emissor** **não sabe** o que aconteceu no **receptor**!
 - Não se pode só retransmitir: **há possibilidade de pacotes duplicados.**
 - **O que fazer?**
- ❑ **Emissor** usaria ACKs / NAKs para ACK / NAK do receptor?
 - E se corromper ACK/NAK do remetente?
- ❑ Retransmitir?
 - Pode causar retransmissão de pacote recebido certo!

RDT 2.0 tem uma falha fatal!

O que acontece se ACK ou NACK com erro ?

- ❑ Remetente **não sabe** o que aconteceu no receptor!
- ❑ Não se pode apenas retransmitir: há possibilidade de pacotes duplicados.

O que fazer?

- ❑ Remetente usa ACKs/NAKs p/ ACK/NAK do receptor?
 - E se perder ACK/NAK do remetente?
- ❑ Retransmitir?
 - Mas pode causar retransmissão de pacote recebido certo!

Lidando com duplicação:

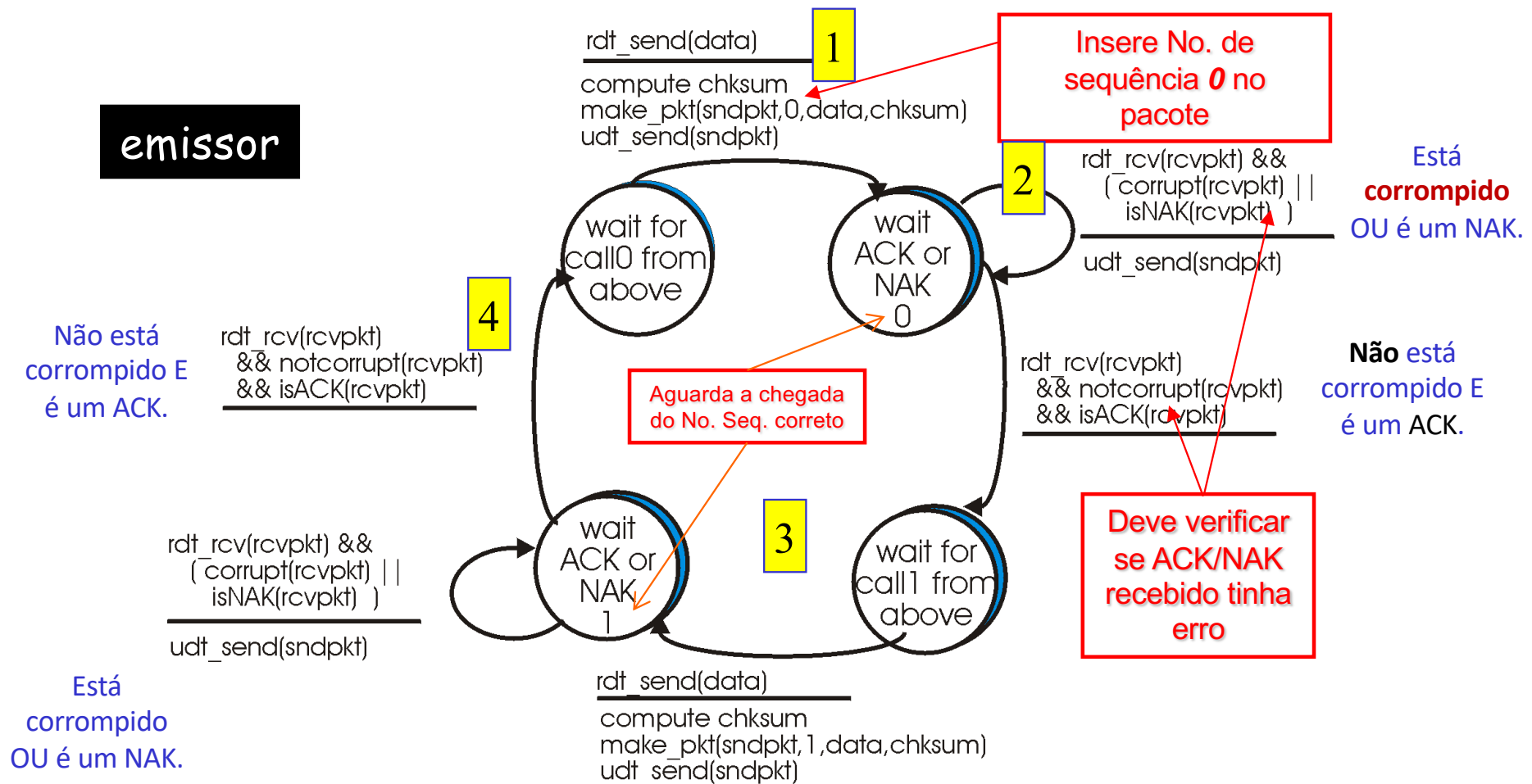
- ❑ Emissor inclui **número de sequência** do pacote.
- ❑ Emissor **retransmite pacote atual** se ACK / NAK for recebido com erro.
- ❑ Receptor **descarta** pacote duplicado.

= RDT 2.1

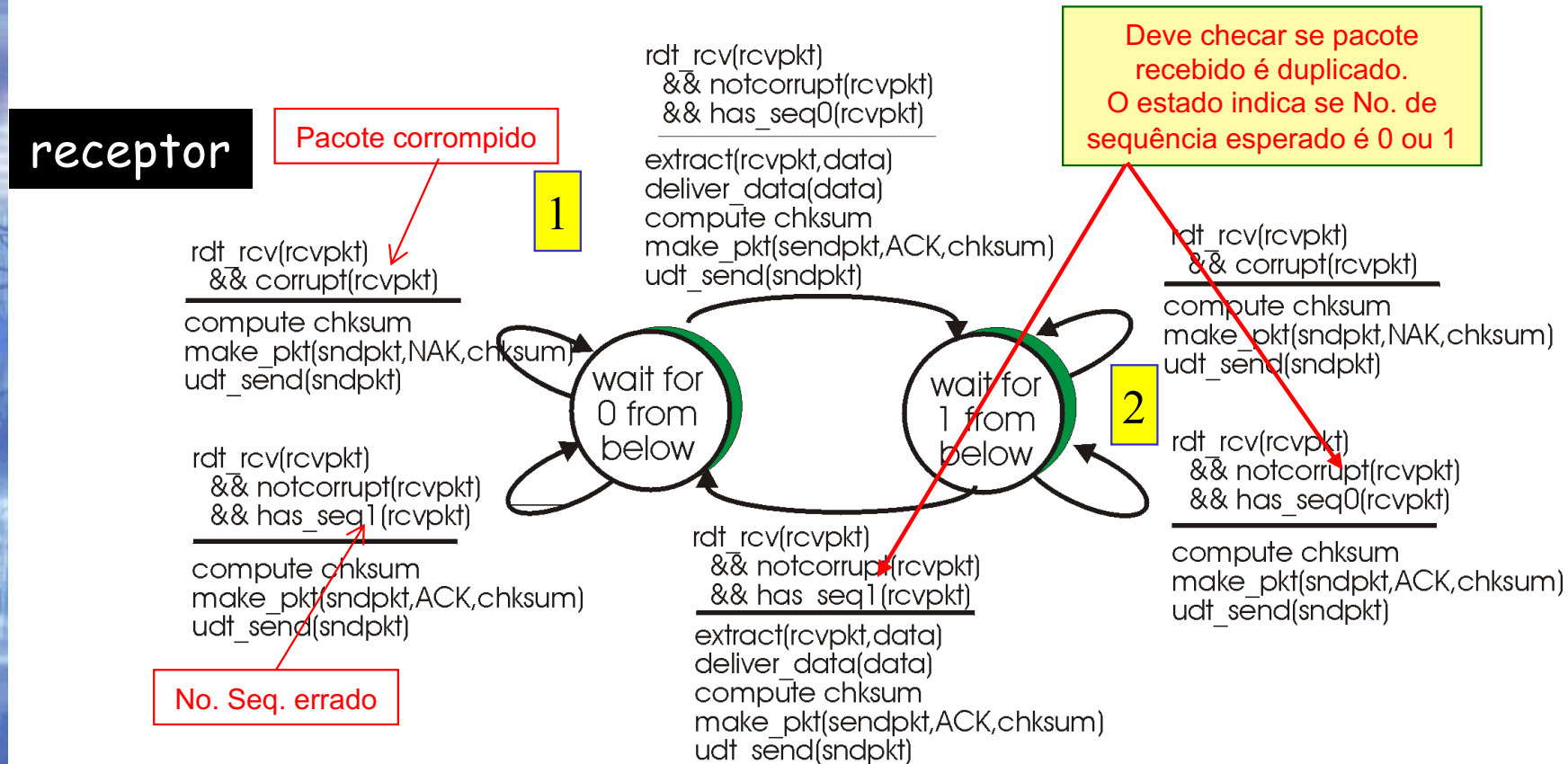
Stop and wait

Remetente envia um pacote, e então aguarda resposta do receptor.

RDT 2.1: EMISSOR – como trata ACK/NAKs com erro



RDT 2.1: RECEPTOR – como trata ACK/NAKs com erro



RDT 2.1: discussão (1)

EMISSOR:

- ❑ Insere **No. de sequência (*seq#*)** no pacote.
- ❑ Bastam dois Nos. de sequência (0 ou 1).
- ❑ Deve verificar se ACK/NAK recebido tem erro.
- ❑ **Duplicou o No. de estados** (compare com RDT 2.0):
 - Tem que “lembrar” se o pacote “atual” tem o número de sequência 0 ou 1.
 - Aqui surge uma das primeiras características do TCP: **manutenção de *status* (*statefull protocol*)**.

rdt2.1: discussão (2)

RECEPTOR:

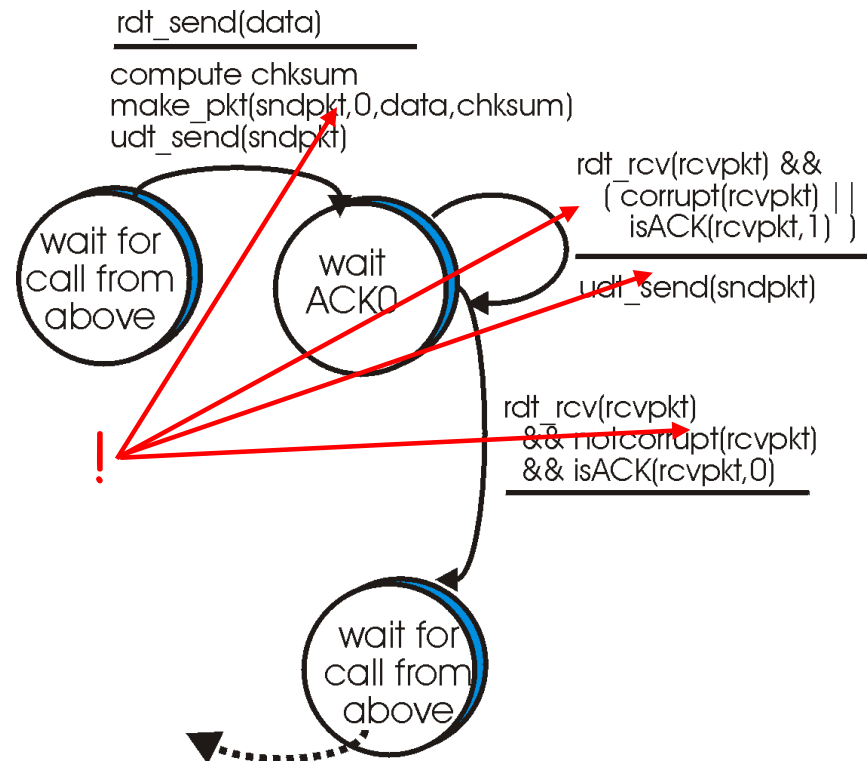
- ❑ Deve checar se pacote recebido é duplicado.
 - **Estado** indica se **No. de sequência esperado é 0 ou 1**.
- ❑ **Receptor** não tem como saber se **último ACK/NAK** foi recebido bem pelo **emissor**.
 - Ele não tem como ter contato direto com o emissor, a não ser por intermédio da indicação de pacotes trocados.

Exercício: RDT 2.2: um protocolo sem NAKs

Fica como **exercício** fazer esta abstração sem NAK

- ❑ Mesma funcionalidade que RDT 2.1, **mas apenas com ACKs.**
- ❑ Ao invés de NAK, **receptor** envia ACK **para o último pacote recebido corretamente.**
 - **Receptor deve incluir explicitamente o No. de sequência do pacote confirmado.**
- ❑ ACK duplicado no **emissor** resulta na mesma ação que o NAK: **retransmite pacote atual.**

EMISSOR



RDT 3.0: canal com erros e perdas (1)

Nova suposição:

Agora, além de corromper dados, o canal também pode **perder pacotes.**

- Pode perder dados, e também perder ACKs.
- Só vamos usar **ACKs** e não mais NAKs.

❑ **Como lidar com perdas?**

- **Como detectar perda de pacotes ?**
- **O que fazer** quando pacotes são perdidos?

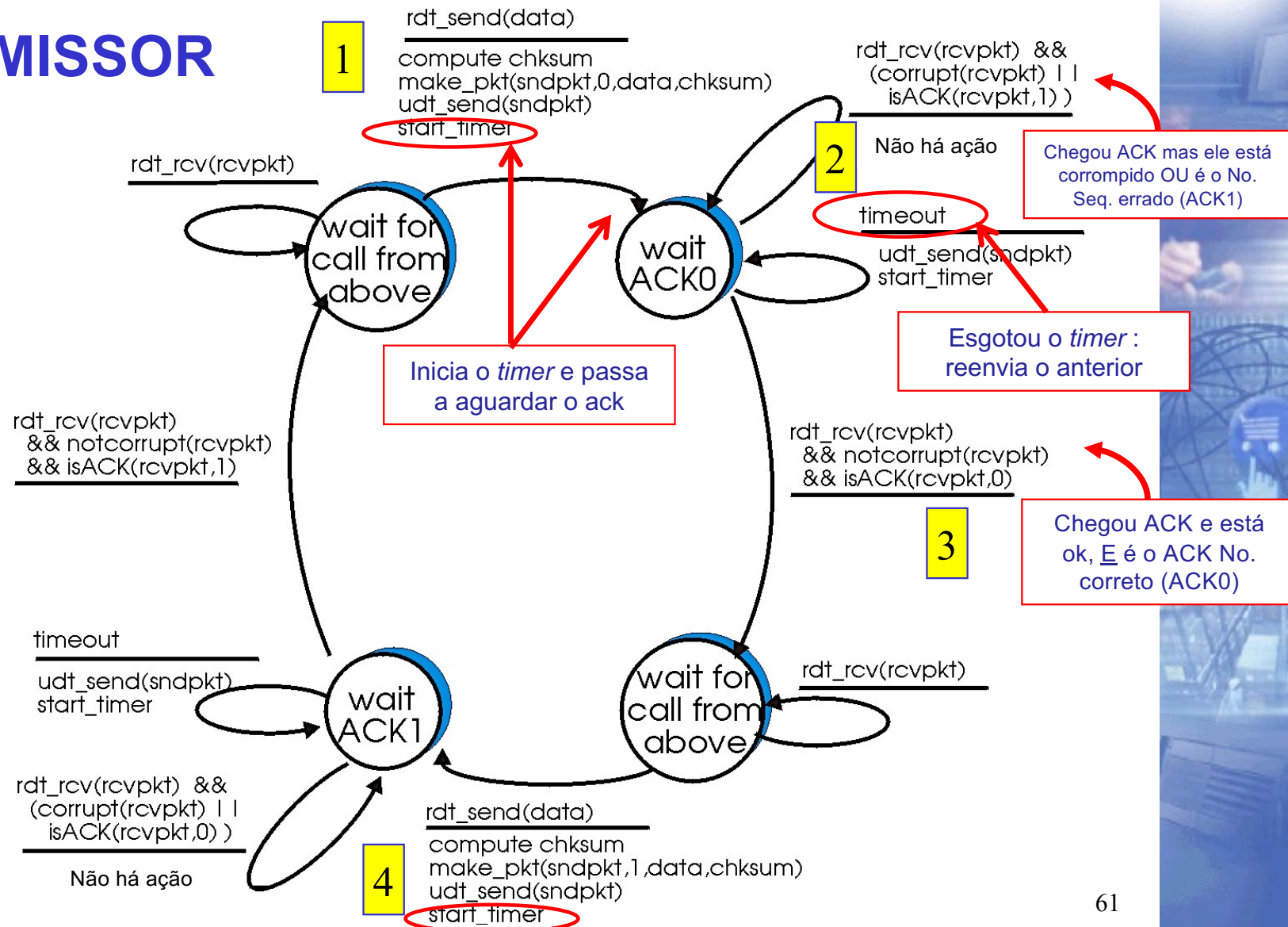
Checksum, No. de sequência, ACKs, e retransmissões vão ajudar, mas não serão suficientes.

rdt3.0: canais com erros e perdas (2)

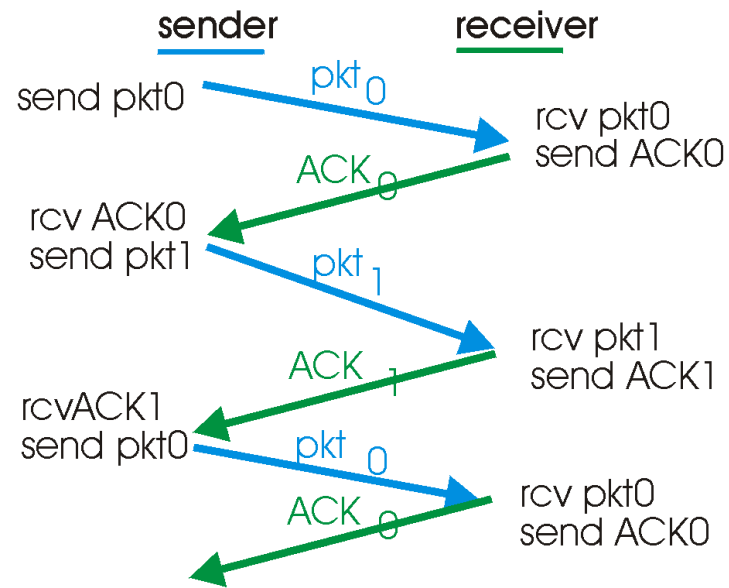
- ❑ **Solução: emissor** aguarda o ACK **que está em trânsito**, por um determinado tempo.
- ❑ Exige uso de **temporizadores (timers)**.
 - **Se há evento de *Timeout*** (esgotamento do *timer*):
 - Retransmite, se nenhum ACK for recebido neste intervalo de tempo.
- ❑ Problema: se pacote (ou o ACK) **está apenas atrasado** (não perdido):
 - **A retransmissão será duplicada.**
 - Mas o uso de número de sequência resolve este problema.
 - **Receptor** deve **avisar o número de sequência** do pacote que está sendo **confirmado**.

RDT 3.0: EMISSOR

emissor

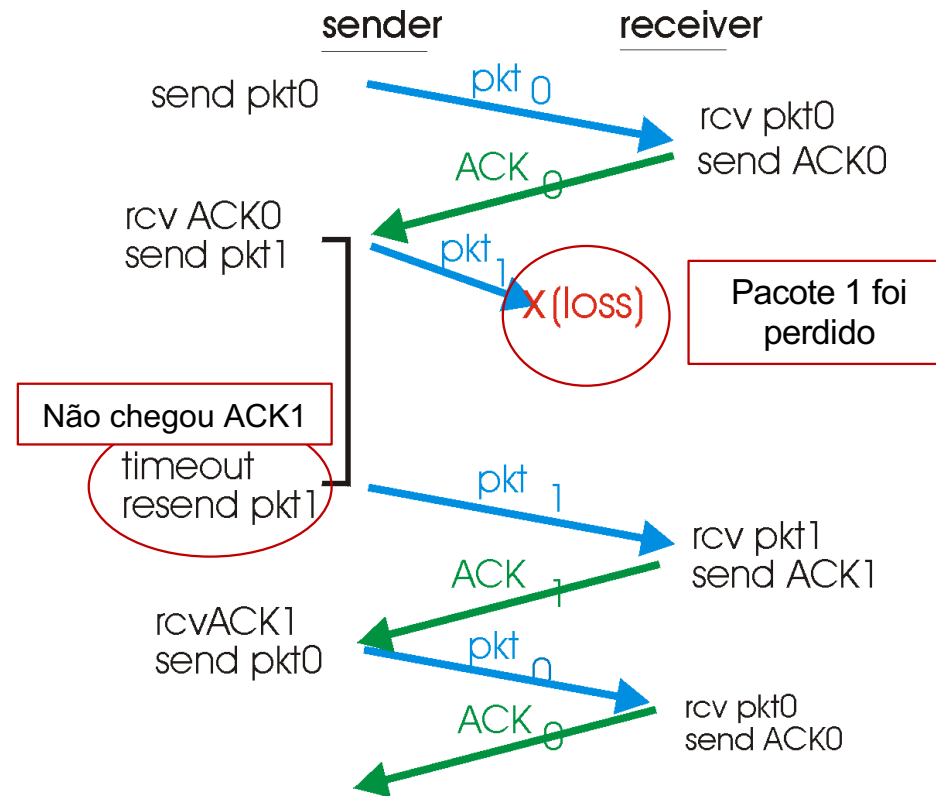


RDT 3.0 em ação: operação normal e perda de pacote



(a) operation with no loss

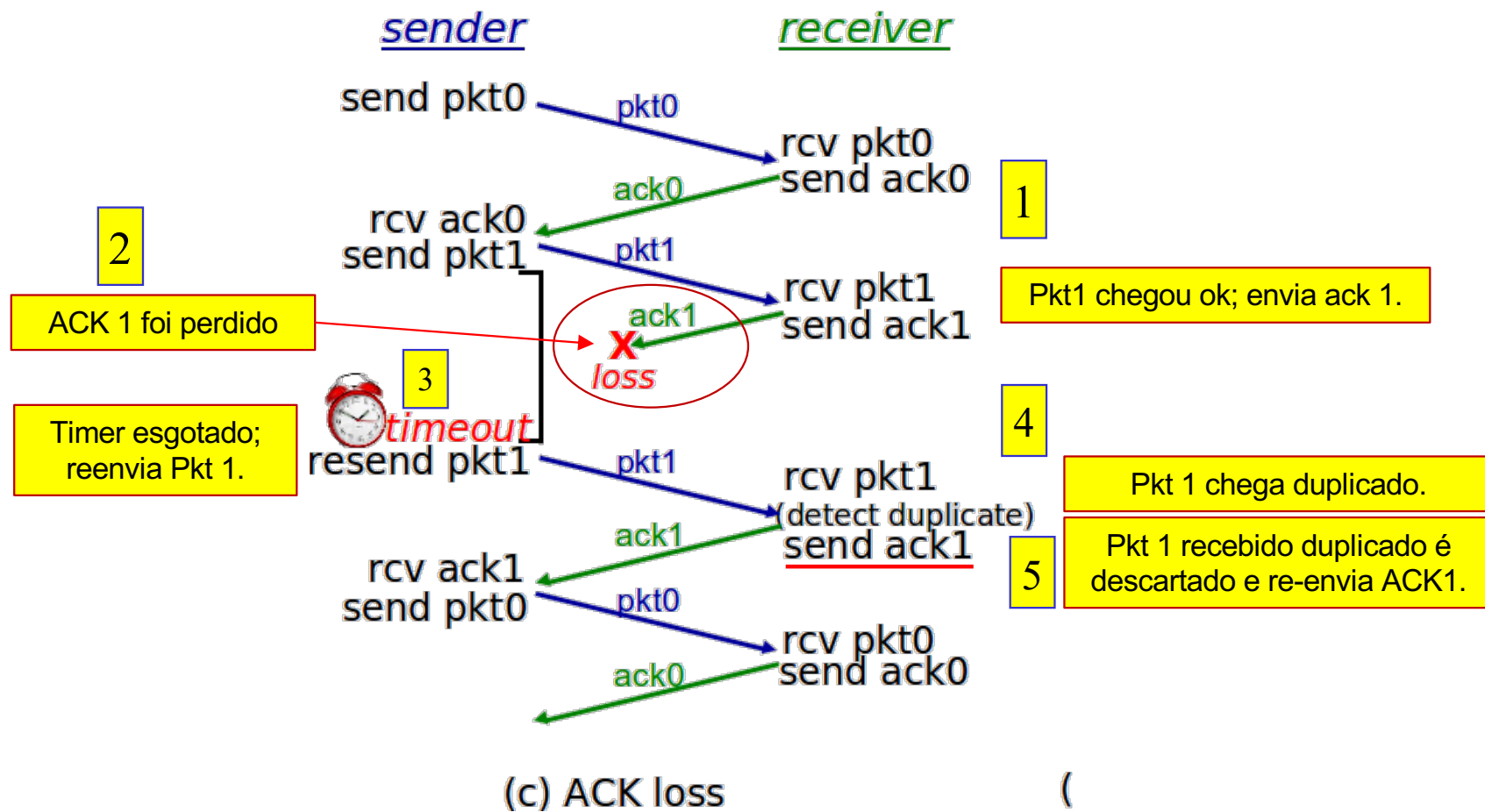
Operação sem perdas



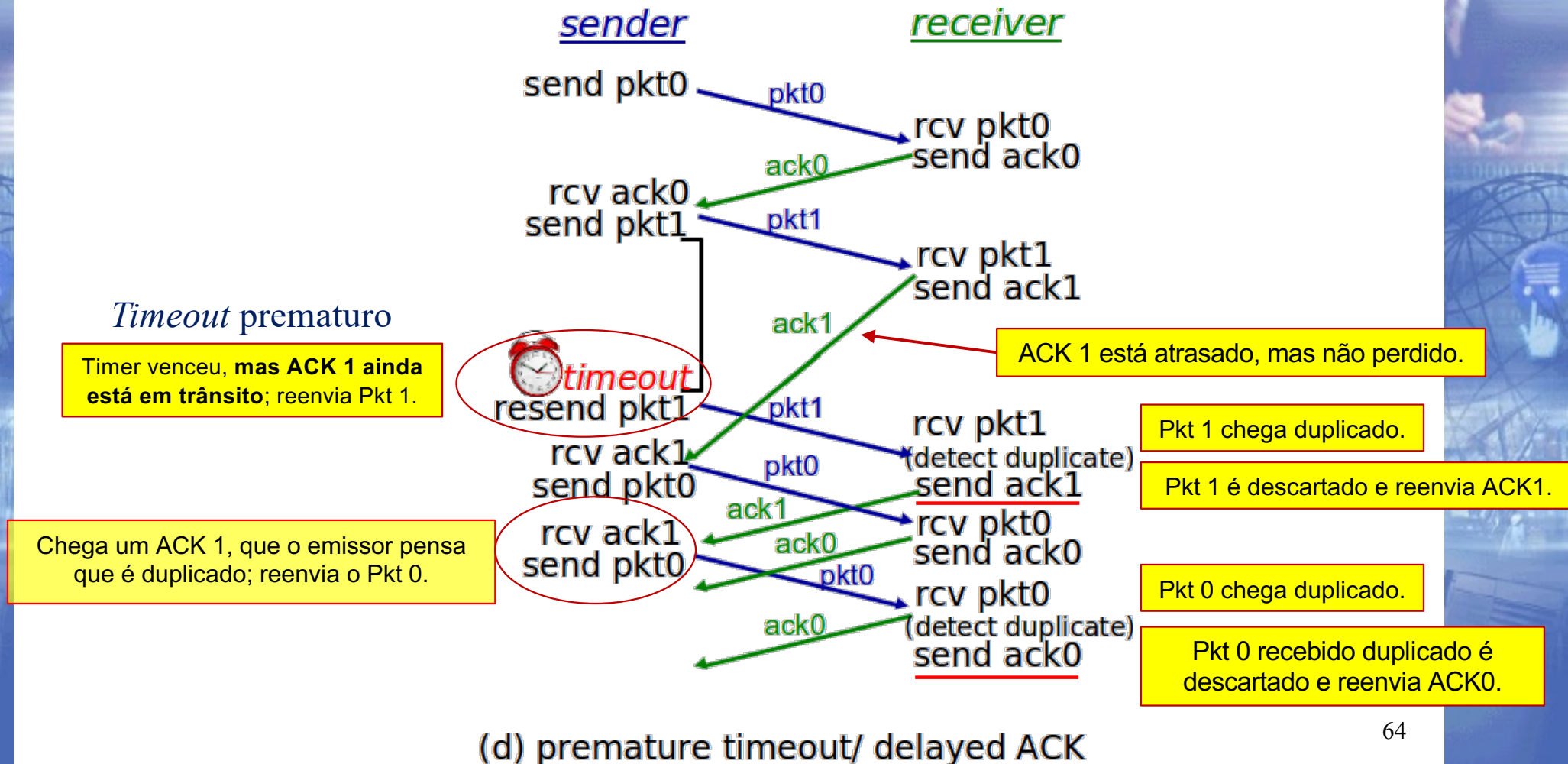
(b) lost packet

Pacote 1 perdido

RDT 3.0 em ação: perda de ack(1)



RDT 3.0 em ação: ack atrasado / *timeout* prematuro



RDT – últimas palavras

- ❑ Terminamos os modelos da criação dos protocolos RDT, e das ideias que os originaram.
 - Serão a **base** para o TCP, **mas ainda não são o TCP**.
- ❑ Em seguida veremos os **protocolos dutados**, que se **juntarão ao RDT para formar o TCP**.

É importante entender bem as ideias do RDT, para poder prosseguir.



Curso de Redes de Computadores

Adriano Mauro Cansian
adriano.cansian@unesp.br

Capítulo 3 Camada de Transporte **(Fim da Parte 1)**

Apêndice: Funções RDT

Função *rdt_send()*

- A função *rdt_send()* na abordagem de *Reliable Data Transfer* (RDT), conforme descrita no livro *Computer Networks* de Kurose **é responsável por iniciar o processo de envio de dados de forma confiável do lado do remetente**, abstraindo a interação entre a camada de aplicação e a camada de transporte subjacente.
- O papel da *rdt_send()* **é servir como o ponto de entrada para a entrega confiável de dados**, permitindo que a aplicação envie informações enquanto o protocolo RDT gerencia as complexidades da comunicação sobre um canal não confiável.
- A *rdt_send()* **é invocada pela camada de aplicação no lado do remetente** quando há dados a serem enviados ao destinatário.
- Seu papel principal **é receber esses dados da aplicação e passá-los para o protocolo RDT, que então os encapsula e os envia através de um canal potencialmente não confiável** (como a camada de rede que pode perder, corromper ou reordenar pacotes).
- A função atua como uma interface que **permite à aplicação solicitar a entrega confiável dos dados**, enquanto o protocolo RDT subjacente lida com os detalhes de garantir que os dados cheguem corretamente e na ordem esperada ao destinatário.

Função `udt_send()`

- O papel da função `udt_send()` é **enviar os dados (ou pacotes) através do canal não confiável em direção ao destinatário**.
- `udt_send()` é a função que **executa a transmissão real dos pacotes pelo canal não confiável**, sendo uma peça essencial para que o protocolo RDT possa construir confiabilidade sobre uma base inerentemente não confiável.
- Ela é **chamada pelo protocolo RDT no lado do remetente para transmitir os pacotes que foram preparados** (por exemplo, com números de sequência, *checksums* ou outros campos necessários) para o lado receptor.
- A `udt_send()` atua como a **interface entre o protocolo RDT e o canal subjacente**, que pode perder, corromper ou reordenar pacotes.

Observação: Enquanto `rdt_send()` é uma função de nível superior que a camada de aplicação usa para iniciar a transferência confiável de dados, `udt_send()` é uma **função de nível inferior usada pelo próprio protocolo RDT para interagir com o canal não confiável**. Em outras palavras:

- `rdt_send()` : Interface entre a aplicação e o protocolo RDT.
- `udt_send()` : Interface entre o protocolo RDT e o canal de comunicação subjacente.

Função *rdt_rcv()*

- Responsável por processar os pacotes recebidos do canal não confiável e gerenciar a entrega confiável dos dados à camada de aplicação no destino.
- *rdt_rcv()* é o ponto de entrada no lado do receptor para lidar com pacotes recebidos, garantindo que os dados sejam validados, organizados e entregues à aplicação de forma confiável.
- *rdt_rcv()* é chamada sempre que um pacote chega ao receptor através do canal, geralmente após ser transmitido pelo remetente via *udt_send()*.
- A função analisa o pacote recebido e executa as ações necessárias para garantir que os dados sejam entregues corretamente e na ordem esperada à aplicação.

Conexão com *udt_send()*: O pacote processado por *rdt_rcv()* no receptor foi originalmente enviado pelo remetente usando *udt_send()* através do canal não confiável.

Contraponto a *rdt_send()*: Enquanto *rdt_send()* inicia a transferência no remetente, *rdt_rcv()* finaliza o processo no receptor, completando o ciclo de comunicação confiável.

Função *deliver_data()*

- A ***deliver_data()*** é invocada pelo protocolo RDT no receptor, geralmente dentro da função *rdt_rcv()*, quando um pacote recebido contém dados válidos e está pronto para ser entregue à aplicação.
 - Seu objetivo é atuar como uma interface entre o protocolo RDT e a camada de aplicação, abstraindo os detalhes do transporte confiável e fornecendo os dados em sua forma utilizável.
 - A função *deliver_data()* é o último passo no processo de recepção do protocolo RDT, responsável por transferir os dados válidos e corretamente ordenados do pacote recebido para a camada de aplicação no receptor.
 - Ela encapsula a entrega final, garantindo que a aplicação receba os dados de forma transparente, sem precisar lidar com as complexidades de erros, perdas ou reordenação.
- **Entrega Final:** Após o protocolo RDT verificar a integridade do pacote (usando *checksums*), confirmar a ordem correta (usando números de sequência, se aplicável) e realizar outras ações necessárias (como enviar ACKs), ***deliver_data()*** pega os dados contidos no pacote e os entrega à aplicação.
 - **Simplicidade:** A função não implementa lógica de controle ou confiabilidade por si só — essa responsabilidade pertence ao protocolo RDT. Ela simplesmente **"finaliza" o processo, transferindo os dados para a camada superior.**

Fim do Apêndice